

UNIVERSIDADE FEDERAL DE SANTA CATARINA
CENTRO TECNOLÓGICO
DEPARTAMENTO DE INFORMÁTICA E ESTATÍSTICA
CURSO DE GRADUAÇÃO EM CIÊNCIA DA COMPUTAÇÃO

Cainã Rinaldi Esteche

Uma Biblioteca de Simulação de Circuitos Digitais em Python com Propagação Reativa

Florianópolis
2026

Cainã Rinaldi Esteche

Uma Biblioteca de Simulação de Circuitos Digitais em Python com Propagação Reativa

Trabalho de Conclusão de Curso apresentado ao curso de Ciência da Computação da Universidade Federal de Santa Catarina como requisito parcial para obtenção do título de Bacharel em Ciência da Computação.

Universidade Federal de Santa Catarina – UFSC

Orientadora Profa. Dra. Cristina Meinhardt
Coorientador Prof. Dr. Ismael Seidel

Florianópolis
2026

Resumo

Estudantes de Ciência da Computação frequentemente enfrentam uma transição abrupta entre a programação em Python, aprendida na disciplina de Programação Orientada a Objetos, e a descrição de hardware em VHDL, exigida logo nos primeiros contatos com a disciplina de Circuitos e Técnicas Digitais. Essa transição é dificultada pela sintaxe formal e pelo modelo de execução concorrente das HDLs, por ambientes de desenvolvimento pesados e por tempos de compilação/síntese elevados, barreiras que se somam à dificuldade dos próprios conceitos de circuitos digitais. Este trabalho propõe um simulador de circuitos digitais implementado inteiramente em Python, sem dependências externas, baseado em um modelo de **propagação reativa de sinais** construído sobre o padrão de projeto *Observer*: cada componente do circuito (sinal, porta lógica, somador, registrador) notifica automaticamente os componentes conectados a ele sempre que sua saída muda, propagando mudanças em cadeia de forma análoga à de um circuito físico. O simulador foi organizado em pacotes que separam representação de sinais, portas lógicas elementares e componentes compostos, incluindo um somador completo (*full adder*) e um somador de N bits (*ripple-carry adder*) construídos inteiramente a partir de portas lógicas, um somador alternativo baseado em conversão para inteiro (que demonstra a possibilidade de misturar Python "puro" com o restante do circuito reativo) e um registrador sensível a borda de *clock*, tanto em uma versão abstrata em lógica Python (`BehavioralRegister`) quanto em uma versão construída inteiramente com *latches*. A validação foi realizada comparando a saída do simulador ao comportamento teoricamente esperado para um conjunto de circuitos de exemplo (portas lógicas, meio-somador, somador completo, três contadores binários e um *flip-flop toggle*), verificada também por testes automatizados, incluindo o tratamento do *overflow* de um contador de N bits em suas duas formas: truncamento explícito no somador baseado em inteiros e propagação natural via sinal de *carry-out* no somador baseado em portas lógicas. Os resultados obtidos confirmam que a abordagem reproduz corretamente os conceitos introdutórios de circuitos digitais em um ambiente de execução já familiar ao estudante, evidenciando seu potencial como etapa preparatória de aprendizado antes do contato com HDLs tradicionais.

Palavras-chave: Circuitos digitais. Programação reativa. Padrão Observer. Python. Simulação. VHDL.

Abstract

Computer Science students often face an abrupt transition between programming in Python, learned in the Object-Oriented Programming course, and describing hardware in VHDL, required early in the Digital Circuits and Techniques course. This transition is made harder by the formal syntax and concurrent execution model of HDLs, by heavyweight development environments, and by long compilation/synthesis times, barriers that add to the difficulty of the digital circuit concepts themselves. This work proposes a digital circuit simulator implemented entirely in Python, with no external dependencies, based on a **reactive signal propagation** model built on top of the Observer design pattern: each circuit component (signal, logic gate, adder, register) automatically notifies the components connected to it whenever its output changes, propagating changes in a chain analogous to a physical circuit. The simulator is organized into packages that separate signal representation, elementary logic gates, and composite components, including a full adder and an N-bit adder (ripple-carry adder) built entirely from logic gates, an alternative adder based on integer conversion (demonstrating that the simulator allows mixing "plain" Python with the rest of the reactive circuit), and a register sensitive to the clock's rising edge, both as an abstract version written in plain Python logic (`BehavioralRegister`) and as a version built entirely from latches. Validation was performed by comparing the simulator's output to the theoretically expected behavior for a set of example circuits (logic gates, half adder, full adder, three binary counters, and a toggle flip-flop), also checked by automated tests, including the handling of N-bit counter overflow in its two forms: explicit truncation in the integer-based adder and natural propagation via a carry-out signal in the gate-based adder. The results confirm that the approach correctly reproduces introductory digital circuit concepts in an execution environment already familiar to the student, showing its potential as a preparatory learning stage before contact with traditional HDLs.

Keywords: Digital circuits. Reactive programming. Observer pattern. Python. Simulation. VHDL.

Lista de ilustrações

Figura 1 – Símbolos padronizados das portas lógicas NOT, AND, OR e XOR. . . 17
Figura 2 – Diagrama de classes (subconjunto representativo) do contrato reativo comum. 28
Figura 3 – Cadeia de propagação reativa do contador binário realimentado. 29
Figura 4 – Um fio (`BitArray` de 1 *bit*) e um barramento de três fios (`BitArray` de 3 *bits*). 29
Figura 5 – `BitSlice` retirando fios de um barramento e `Bus` agrupando fios em um barramento. 31
Figura 6 – Porta XOR anotada com as linhas do Algoritmo 2. 32
Figura 7 – Somador completo (*full adder*) montado com portas lógicas, com a linha do Algoritmo 3 correspondente a cada porta. 33
Figura 8 – `RippleCarryAdder` de 3 *bits* formado pelo encadeamento de três `FullAdder`, com as linhas do Algoritmo 4. 34
Figura 9 – `BehavioralAdder` como bloco comportamental, com as linhas dos Algoritmos 5 e 6. 37
Figura 10 – `BehavioralRegister` representado como registrador, com as linhas do Algoritmo 7. 39
Figura 11 – *Latch* SR de portas NOR cruzadas, com as linhas do Algoritmo 8. . . . 41
Figura 12 – *Latch* D montado sobre um *latch* SR, com as linhas do Algoritmo 9. . . 42
Figura 13 – *Flip-flop* D mestre-escravo, com as linhas do Algoritmo 10. 43
Figura 14 – `DFlipFlopRegister` de 3 *bits*, com as linhas do Algoritmo 11. 45
Figura 15 – Diagrama de sequência de um pulso de *clock* no *flip-flop toggle*. 49

Lista de tabelas

Tabela 1 – Tabelas-verdade das portas lógicas básicas. 16
Tabela 2 – Comparação entre trabalhos relacionados e o simulador proposto. . . . 25
Tabela 3 – Propagação reativa através de uma porta NOT. 51
Tabela 4 – Saída do meio-somador e do somador completo comparadas à tabela-verdade esperada. 52
Tabela 5 – Saída do contador de 3 *bits* e do *flip-flop toggle* ao longo de sucessivos pulsos de *clock*. 53
Tabela 6 – Contador de 3 *bits* montado com RippleCarryAdder, com carry_out. . 54
Tabela 7 – Validação do *latch* SR (NOR cruzado). 55
Tabela 8 – Saída do *flip-flop toggle* montado com DFlipFlop. 56
Tabela 9 – Contador de 3 *bits* com BehavioralRegister comparado ao mesmo contador com DFlipFlopRegister. 56
Tabela 10 – Recursos de observação e depuração em diferentes ferramentas. 62

Lista de Quadros

1 Log de propagação de um pulso de *clock* no *flip-flop toggle*. 53
2 Validação do `DFlipFlop` isolado. 55
3 Saídas observadas durante a propagação do Exemplo 6. 58
4 Atividade de chaveamento medida pelo Exemplo 7. 59
5 Saída do Exemplo 8. 60
6 Caminho da propagação impresso pelo Exemplo 9. 61

Lista de Algoritmos

1 Classe BitArray (signals/bit_array.py). 30
2 Porta Xor (gates/xor_gate.py). 32
3 FullAdder (components/full_adder.py). 33
4 RippleCarryAdder (components/ripple_carry_adder.py). 34
5 BehavioralAdder (components/behavioral_adder.py). 35
6 Funções auxiliares de conversão e *padding* binário (utils/binary_conversion.py). 36
7 Classe BehavioralRegister (components/behavioral_register.py). 38
8 SRLatch (components/sr_latch.py). 40
9 DLatch (components/d_latch.py). 42
10 DFlipFlop (components/d_flip_flop.py). 43
11 DFlipFlopRegister (components/d_flip_flop_register.py). 44

Lista de Exemplos

1 Meio-somador composto a partir de Xor e And. 45
2 Contador binário de N *bits* com BehavioralAdder. 46
3 Contador binário de N *bits* com somador de portas lógicas. 47
4 Contador binário de N *bits* com registrador real (DFlipFlopRegister). . . . 47
5 *Flip-flop toggle*. 48
6 Observação dos estados transitórios de um somador. 58
7 Contagem da atividade de chaveamento de um somador de 8 *bits*. 59
8 Sonda que detecta a combinação inválida do *latch* SR. 60
9 Reconstrução do caminho percorrido por uma mudança de sinal. 61
10 Verificação exaustiva de equivalência entre as descrições estrutural e comportamental. 62

Lista de Abreviaturas e Siglas

ASIC *Application-Specific Integrated Circuit* 18

FPGA *Field-Programmable Gate Array* 12, 18

HDL *Hardware Description Language* 12, 18

IDE *Integrated Development Environment* 12

POO Programação Orientada a Objetos 12

UFSC Universidade Federal de Santa Catarina 12

ULA Unidade Lógica e Aritmética 64

VHDL *VHSIC Hardware Description Language* 12

Sumário

Lista de ilustrações 4

Lista de tabelas 5

Sumário 10

1 INTRODUÇÃO 12
1.1 Problema 14
1.2 Objetivos 14
1.3 Justificativa 15
1.4 Estrutura do Trabalho 15

2 FUNDAMENTAÇÃO TEÓRICA 16
2.1 Circuitos Digitais 16
2.1.1 Álgebra Booleana e Portas Lógicas 16
2.1.2 Circuitos Sequenciais, *Clock* e Registradores 17
2.1.3 Somadores Binários 17
2.2 Linguagens de Descrição de *Hardware* 18
2.3 Programação Reativa e o Padrão *Observer* 19

3 TRABALHOS RELACIONADOS 21
3.1 Simuladores Gráficos e Híbridos de Circuitos Digitais 21
3.2 Bibliotecas e HDLs Embutidas em Python 22
3.3 Iniciativas Pedagógicas para HDLs Tradicionais 23
3.4 Síntese Comparativa e Lacuna Identificada 24

4 PROPOSTA 27
4.1 Visão Geral da Proposta 27
4.2 Visão Geral da Arquitetura 28
4.3 Representação de Sinais 29
4.3.1 `BitArray` 29
4.3.2 `BitSlice` e `Bus` 30
4.4 Portas Lógicas 31
4.5 Somador com Portas Lógicas 33
4.6 `BehavioralAdder`: um Somador Comportamental em Python 35
4.7 Componente Sequencial: `BehavioralRegister` 37
4.8 Registrador Real com *Latches* 39

4.8.1 *Latch* SR . . . 40
4.8.2 *Latch* D e *Flip-Flop* D Mestre-Escravo . . . 41
4.8.3 `DFlipFlopRegister`: um Registrador de N *Bits* Real . . . 44
4.9 Composição de Circuitos . . . 45
4.9.1 Meio-Somador . . . 45
4.9.2 Somador Completo . . . 46
4.9.3 Contador Binário Realimentado com `BehavioralAdder` . . . 46
4.9.4 Contador Binário Realimentado com Portas Lógicas . . . 46
4.9.5 Contador Binário Realimentado com Registrador Real . . . 47
4.9.6 *Flip-Flop Toggle* . . . 47
4.10 Tecnologias Utilizadas . . . 48
4.11 A *Engine* de Propagação . . . 48

5 RESULTADOS E DISCUSSÃO . . . 51
5.1 Validação dos Circuitos Implementados . . . 51
5.1.1 Porta NOT Reativa . . . 51
5.1.2 Meio-Somador e Somador Completo . . . 51
5.1.3 Registrador Simples . . . 52
5.1.4 Contador Binário e *Flip-Flop Toggle* . . . 52
5.1.5 Contador Binário Montado Só com Portas Lógicas . . . 54
5.1.6 Registrador Real com *Latches* . . . 54
5.2 Testes Automatizados . . . 57
5.3 Recursos de Observação e Depuração . . . 57
5.3.1 Estados Transitórios (*Glitches*) . . . 58
5.3.2 Atividade de Chaveamento . . . 59
5.3.3 Sondas com Asserções . . . 59
5.3.4 Caminho da Propagação . . . 60
5.3.5 Equivalência entre Descrições . . . 61
5.3.6 Comparação com Outras Ferramentas . . . 62
5.4 Limitações da Abordagem . . . 63
5.5 Discussão . . . 63

6 CONCLUSÃO . . . 65
6.1 Contribuições . . . 65
6.2 Trabalhos Futuros . . . 66

REFERÊNCIAS . . . 68

1 Introdução

Circuitos digitais formam a base de praticamente todos os sistemas computacionais modernos, e seu estudo é parte obrigatória da formação em Ciência da Computação. No currículo do curso de Ciência da Computação da Universidade Federal de Santa Catarina (UFSC), esse conteúdo é introduzido na disciplina de Circuitos e Técnicas Digitais, na qual os estudantes têm o primeiro contato formal com portas lógicas, álgebra booleana, circuitos combinacionais e sequenciais, e com uma *Hardware Description Language* (HDL), tipicamente *VHSIC Hardware Description Language* (VHDL).

Paralelamente, é comum que esses mesmos estudantes estejam cursando, no mesmo período ou em período próximo, a disciplina de Programação Orientada a Objetos (POO), que utiliza Python como linguagem principal. O resultado é que o aluno transita, dentro de um curto intervalo de tempo, entre dois paradigmas de programação extremamente distintos: de um lado, Python, uma linguagem imperativa, sequencial, de sintaxe enxuta e com um ciclo de desenvolvimento quase instantâneo (editar, executar, ver o resultado); de outro, VHDL, uma linguagem declarativa e concorrente, com sintaxe verbosa e formal, que descreve *hardware* e não um programa a ser executado passo a passo.

A motivação deste trabalho nasce justamente dessa distância entre as duas experiências de aprendizado vividas pelo estudante de Ciência da Computação. Observam-se, na prática, algumas dificuldades recorrentes:

- **Distância de paradigma e de sintaxe**: enquanto em Python o aluno já está habituado a um código curto e direto, VHDL exige a declaração explícita de entidades, arquiteturas, sinais, tipos e processos antes mesmo de descrever a lógica desejada. Essa sintaxe extensa e formal, combinada com um modelo de execução concorrente pouco intuitivo para quem só programou de forma sequencial, torna os primeiros contatos com a disciplina de Circuitos e Técnicas Digitais particularmente custosos;

- **Ambientes de desenvolvimento pesados e pouco ágeis**: as *Integrated Development Environments* (IDEs) tradicionalmente utilizadas para VHDL (como Quartus, da Intel, ou Vivado, da Xilinx/AMD) são ferramentas robustas, voltadas para síntese em *Field-Programmable Gate Arrays* (FPGAs) reais, porém pesadas, lentas para abrir e configurar, e com uma curva de aprendizado própria que é somada à da linguagem em si;

- **Tempo de compilação/síntese elevado**: diferentemente do ciclo quase instantâneo de "salvar e rodar" ao qual o aluno está acostumado em Python, a simulação ou síntese de um circuito VHDL pode levar segundos a minutos, mesmo para exemplos simples. Esse tempo de espera atrapalha a experimentação rápida e a construção de

intuição sobre o comportamento do circuito, que é justamente o tipo de aprendizado que se beneficia de iteração rápida e *feedback* imediato;

- **O problema não é exclusivo do VHDL**: linguagens de descrição de *hardware* aparentadas, como Verilog e SystemVerilog, compartilham a mesma natureza concorrente, sintaxe formal e ecossistema de ferramentas pesado, de modo que a dificuldade relatada aqui não é uma particularidade do VHDL, mas sim das HDLs tradicionais como classe de linguagem.

A busca por ferramentas que tornem os conteúdos de *hardware* mais acessíveis a estudantes de Computação não é recente. Há mais de duas décadas, Hacker e Sitte (2004) propuseram um ambiente interativo para o ensino de lógica digital combinacional e sequencial, motivados pela falta de *software* de apoio adequado a esse ensino. Mais recentemente, Garay et al. (2019) recorreram à visualização de simulações em VHDL para ensinar um tópico de Arquitetura e Organização de Computadores que, na experiência dos autores, é bastante complexo para os estudantes, evidenciando que mesmo o VHDL tem sido usado como meio, e não como fim, em iniciativas que buscam reduzir barreiras de aprendizado.

Na comunidade brasileira de educação em computação, dois mapeamentos sistemáticos recentes sintetizam parte dessa produção. Santini et al. (2023) identificaram treze jogos educacionais voltados ao ensino de Circuitos Digitais publicados nos últimos dez anos, partindo da constatação de que "a disciplina de Circuitos Lógicos muitas vezes é considerada tediosa pelos alunos" (SANTINI et al., 2023, p. 814); em relato de experiência anterior, Carvalho e Nakamura (2013) já haviam observado falta de interesse dos alunos pela disciplina, vista por eles como um assunto próprio da Engenharia. Já Dôndici et al. (2025, p. 240) abrem seu mapeamento afirmando que "o ensino de Circuitos Lógicos, embora essencial nos cursos de Computação, enfrenta desafios pela complexidade dos conceitos". A complexidade também aparece em tópicos mais específicos da área: Jurgina et al. (2021, p. 191) observam que a complexidade do tema confiabilidade de circuitos "é uma barreira para que seu estudo seja incorporado em sala de aula", o que motivou o desenvolvimento de uma ferramenta dedicada a esse ensino. Na literatura internacional, Haase (2022) propõe uma ferramenta de simulação interativa de circuitos digitais, executada no navegador, para apoiar o modelo de sala de aula invertida e o estudo autônomo. A dificuldade também se reflete em dados de evasão: em um estudo sobre os cursos de Computação da Universidade Estadual de Maringá, Fukao et al. (2023) estimaram taxas de evasão acumuladas de 47,6% em Ciência da Computação e de 54,5% em Informática para os ingressantes de 2005 a 2021, e os estudantes evadidos apontaram a didática adotada pelos professores como o principal fator percebido como impactante para a evasão. Em conjunto, essas evidências situam a dificuldade aqui relatada como um problema reconhecido e ativamente pesquisado há pelo

menos duas décadas, tanto na literatura internacional quanto na comunidade brasileira de educação em computação.

Diante desse cenário, este trabalho parte da hipótese de que um simulador de circuitos digitais implementado em Python, linguagem com a qual o estudante já possui familiaridade prévia por meio da disciplina de POO, pode servir como uma etapa intermediária de aprendizado, permitindo que os conceitos fundamentais de portas lógicas, propagação de sinais e componentes sequenciais (como registradores) sejam compreendidos em um ambiente leve, de execução imediata e sintaxe já conhecida, antes que o aluno enfrente a barreira adicional imposta pelas HDLs.

1.1 Problema

Como apoiar o aprendizado dos conceitos básicos de circuitos digitais (portas lógicas, propagação de sinais e componentes sequenciais) de forma que o estudante de Ciência da Computação possa se concentrar na lógica do circuito em si, sem que a sintaxe, o ambiente de desenvolvimento e o tempo de compilação de uma HDL tradicional se tornem obstáculos adicionais nesse primeiro contato?

1.2 Objetivos

O objetivo geral deste trabalho é desenvolver, em Python, um simulador didático de circuitos digitais baseado em propagação reativa de sinais, capaz de apoiar o aprendizado dos conceitos introdutórios da disciplina de Circuitos e Técnicas Digitais.

Os objetivos específicos são:

- Modelar uma abstração de barramento de *bits* (sinal), capaz de notificar automaticamente os componentes conectados a ele quando seu valor muda;
- Implementar as portas lógicas básicas (NOT, AND, OR, XOR) como componentes que recalculam sua saída automaticamente a partir da propagação de mudanças em suas entradas;
- Implementar um componente de registrador (*latch*/*flip-flop*) sensível a borda de subida de um sinal de *clock*, permitindo a construção de circuitos sequenciais e de memória;
- Demonstrar, a partir desses componentes básicos, a construção de circuitos compostos (por exemplo, um meio-somador, um contador binário e um *flip-flop* do tipo *toggle*);
- Avaliar, por meio desses circuitos de exemplo, se a abordagem proposta reproduz corretamente o comportamento esperado dos circuitos digitais correspondentes.

1.3 Justificativa

Este trabalho se justifica por atacar um obstáculo concreto observado na trajetória de estudantes de Ciência da Computação: o salto abrupto entre a programação em Python (aprendida em POO) e a descrição de *hardware* em VHDL (exigida em Circuitos e Técnicas Digitais). Ao propor uma ferramenta que utiliza uma linguagem já familiar ao aluno para modelar os mesmos conceitos (portas lógicas, propagação de sinais, registradores), pretende-se reduzir a carga cognitiva do primeiro contato com a disciplina, isolando a dificuldade dos conceitos de circuitos digitais da dificuldade adicional imposta pela sintaxe e pelas ferramentas de uma HDL.

Além do valor pedagógico, o desenvolvimento do simulador também tem valor técnico: sua implementação exige a aplicação de conceitos de engenharia de *software* (como o padrão de projeto *Observer* e a programação orientada a eventos) para resolver um problema de domínio distinto (simulação de *hardware*), evidenciando a transferência de conceitos entre áreas do curso de Ciência da Computação.

1.4 Estrutura do Trabalho

O restante deste trabalho está organizado da seguinte forma: o Capítulo 2 apresenta a fundamentação teórica necessária, cobrindo circuitos digitais, HDLs e o modelo de programação reativa/padrão *Observer*; o Capítulo 3 discute trabalhos relacionados, incluindo outros simuladores de circuitos digitais e iniciativas pedagógicas para HDLs; o Capítulo 4 descreve a arquitetura e a implementação do simulador; o Capítulo 5 apresenta os circuitos construídos como estudo de caso e discute os resultados obtidos; e o Capítulo 6 traz as considerações finais e sugestões de trabalhos futuros.

2 Fundamentação Teórica

Este capítulo apresenta os conceitos necessários para compreender o restante do trabalho: os fundamentos de circuitos digitais que o simulador precisa reproduzir corretamente (Seção 2.1), as linguagens de descrição de *hardware* que motivam a proposta (Seção 2.2) e o modelo de programação reativa/padrão *Observer* sobre o qual o simulador é construído (Seção 2.3). Trabalhos relacionados, incluindo outros simuladores e iniciativas pedagógicas para HDLs, são discutidos separadamente no Capítulo 3.

2.1 Circuitos Digitais

Circuitos digitais são sistemas que operam sobre sinais discretos, representados por apenas dois níveis lógicos: 0 (falso/baixo) e 1 (verdadeiro/alto). Essa representação binária confere aos sistemas digitais maior robustez a ruído e simplicidade de projeto quando comparados a circuitos analógicos, que operam sobre grandezas contínuas (TOCCI; WIDMER; MOSS, 2011; FLOYD, 2007).

A unidade fundamental de informação é o ***bit***. Um conjunto ordenado de *bits* forma um **barramento** (*bus*); um barramento de n *bits* representa 2^n valores distintos, potencialmente interpretados como um número inteiro sem sinal em notação binária posicional (por exemplo, o barramento de 3 *bits* `101` corresponde ao decimal 5). No simulador desenvolvido neste trabalho, essa noção de barramento é representada diretamente por uma lista de valores booleanos, encapsulada na classe `BitArray` (detalhada no Capítulo 4).

2.1.1 Álgebra Booleana e Portas Lógicas

O comportamento dos circuitos digitais é descrito formalmente pela álgebra booleana, que define operações sobre valores lógicos. As operações elementares (**NOT**, **AND**, **OR** e **XOR**) são implementadas fisicamente por **portas lógicas**, cujo comportamento é resumido pelas tabelas-verdade da Tabela 1.

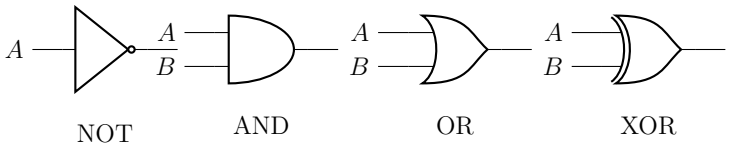
Tabela 1 – Tabelas-verdade das portas lógicas básicas.

A	B	NOT A	A AND B	A OR B	A XOR B
0	0	1	0	0	0
0	1	1	0	1	1
1	0	0	0	1	1
1	1	0	1	1	0

Fonte: elaborada pelo autor (2026), com base na convenção usual de tabelas-verdade (TOCCI; WIDMER; MOSS, 2011).

A Figura 1 apresenta os símbolos tradicionais, de formato distintivo, usados para representar essas quatro portas em diagramas de circuitos, notação empregada nas figuras deste trabalho.

Figura 1 – Símbolos padronizados das portas lógicas NOT, AND, OR e XOR.



Fonte: elaborada pelo autor (2026), com base nos símbolos tradicionais de portas lógicas (TOCCI; WIDMER; MOSS, 2011).

Circuitos mais complexos são obtidos pela composição dessas portas elementares. Quando a saída de um circuito depende apenas dos valores presentes em suas entradas no instante atual, sem depender de estados anteriores, ele é chamado de **circuito combinacional**.

2.1.2 Circuitos Sequenciais, *Clock* e Registradores

Diferentemente dos circuitos combinacionais, **circuitos sequenciais** possuem memória: sua saída depende não apenas das entradas atuais, mas também de um estado interno armazenado. Esse estado é tipicamente mantido em elementos de memória elementares, como ***latches*** e ***flip-flops***, e sua atualização é coordenada por um sinal de ***clock***, um sinal periódico que alterna entre os níveis 0 e 1.

Um ***flip-flop* sensível a borda** atualiza seu estado apenas no instante em que o *clock* transiciona de um nível para outro (por exemplo, na borda de subida, isto é, na transição de 0 para 1), permanecendo estável no restante do tempo. Um **registrador** é, em sua forma mais simples, um *flip-flop* (ou um conjunto deles) que armazena um valor de múltiplos *bits*, capturando o valor presente em sua entrada de dados no instante da borda de *clock*. Esse é exatamente o comportamento reproduzido pela classe `BehavioralRegister` do simulador, em lógica Python, e pelo `DFlipFlopRegister`, em nível de portas lógicas: ambos só atualizam sua saída quando o sinal usado como *clock* transiciona de falso para verdadeiro.

Registradores realimentados, isto é, cuja saída alimenta, direta ou indiretamente, sua própria entrada, são a base para a construção de circuitos com estado persistente, como contadores e acumuladores, que serão usados como estudo de caso neste trabalho.

2.1.3 Somadores Binários

O **meio-somador** (*half adder*) é o circuito combinacional mais simples capaz de somar dois *bits*: sua saída de soma é dada por $A \oplus B$ (uma porta XOR) e sua saída de *carry*

(vai-um) por $A \cdot B$ (uma porta AND). O **somador completo** (*full adder*) estende essa ideia somando também um *carry* de entrada, permitindo o encadeamento de somadores para compor um somador de múltiplos *bits* (*ripple-carry adder*).

Um aspecto relevante para este trabalho é o conceito de ***overflow***: quando a soma de dois números representados em n *bits* excede o maior valor representável nesses n *bits* ($2^n - 1$), o resultado "estoura" a largura do barramento. Em um contador de *hardware* real, esse comportamento se manifesta como um *wraparound*: os *bits* mais significativos que excedem a largura são descartados, e o contador retorna a zero. No simulador, esse comportamento surge naturalmente no somador montado com portas lógicas, mas precisa ser reproduzido explicitamente no somador comportamental, como discutido no Capítulo 4.

2.2 Linguagens de Descrição de *Hardware*

Além da montagem física ou do desenho esquemático, circuitos digitais podem ser projetados por meio de **linguagens de descrição de *hardware*** (HDLs), das quais VHDL e Verilog (e sua evolução, SystemVerilog) são as mais utilizadas tanto na indústria quanto na academia. HDLs permitem descrever o comportamento e a estrutura de um circuito em um nível de abstração mais alto que o esquemático de portas, sendo posteriormente processadas por ferramentas de síntese que as convertem em circuitos configuráveis em FPGAs ou fabricáveis em *Application-Specific Integrated Circuits* (ASICs).

Essas linguagens distinguem dois estilos de descrição de um mesmo componente. Na **descrição estrutural** (*structural*), o componente é definido pelas partes que o compõem e pelas ligações entre elas, como em um esquemático: um somador de N *bits* é descrito como N somadores completos interligados, e cada somador completo como um conjunto de portas. Na **descrição comportamental** (*behavioral*), o componente é definido pelo que ele faz, por meio de construções de alto nível da linguagem (operadores aritméticos, condicionais, processos sensíveis a eventos), sem especificar de quais portas ele é feito: o mesmo somador pode ser descrito simplesmente como `s <= a + b` (ASHENDEN, 2008). Ambos os estilos são usados na prática, muitas vezes no mesmo projeto, e o simulador deste trabalho adota a mesma distinção: os componentes com prefixo `Behavioral` são descrições comportamentais escritas em Python, e cada um tem uma contraparte estrutural montada com portas lógicas (Capítulo 4).

O aspecto central que distingue uma HDL de uma linguagem de programação de propósito geral como Python é o seu **modelo de execução**: enquanto um programa em Python é executado de forma sequencial, instrução após instrução, uma descrição em VHDL ou Verilog representa *hardware*: todos os processos e atribuições concorrentes descritos no código correspondem a componentes que operam **simultaneamente**, reagindo a mudanças em seus sinais de entrada a qualquer momento, e não em uma ordem definida pela posição do código no arquivo-fonte.

Essa diferença de paradigma, somada à sintaxe formal exigida por essas linguagens (declaração explícita de entidades, arquiteturas, sinais tipados, sensibilidade de processos, entre outros elementos) e ao fluxo de trabalho baseado em compilação/síntese, tipicamente mais lento que o ciclo de execução imediata de uma linguagem interpretada, compõe o conjunto de barreiras discutido na motivação deste trabalho (Capítulo 1). SystemVerilog, por sua vez, estende Verilog com recursos adicionais de verificação e modelagem, mas preserva a mesma natureza concorrente subjacente, de modo que compartilha as mesmas barreiras de aprendizado aqui discutidas.

2.3 Programação Reativa e o Padrão *Observer*

Em um circuito digital físico, uma mudança de sinal em um ponto se propaga em cadeia por todas as conexões dependentes, e não instantaneamente: cada porta lógica introduz um pequeno atraso de propagação entre a mudança em sua entrada e a atualização de sua saída. O simulador desenvolvido neste trabalho abstrai esse atraso, tratando a propagação como instantânea (sem noção de tempo simulado); as implicações dessa simplificação são discutidas na Seção 5.1.6. A **programação reativa** é um paradigma de programação orientado à propagação automática de mudanças: um valor reativo notifica automaticamente todos os elementos que dependem dele sempre que muda, sem que o código cliente precise solicitar essa atualização explicitamente (BAINOMUGISHA et al., 2013).

A principal vantagem desse paradigma em relação à programação imperativa convencional está em quem fica responsável por manter os valores dependentes atualizados. No estilo imperativo, cada vez que um valor muda o programador precisa lembrar de recalcular, na ordem certa, tudo o que depende dele, seja chamando funções de atualização espalhadas pelo código, seja consultando periodicamente (*polling*) se algo mudou; esquecer uma dessas chamadas deixa o programa em um estado inconsistente, e a ordem correta de recálculo precisa ser conhecida e mantida manualmente. Na programação reativa, as dependências são declaradas uma única vez, no momento em que os elementos são conectados, e a responsabilidade de propagar as mudanças passa a ser do próprio mecanismo de execução: os dependentes são sempre recalculados a partir dos valores atuais, e apenas o que de fato depende do valor alterado é recalculado (BAINOMUGISHA et al., 2013). Em comparação com um modelo de avaliação explícita, em que o circuito inteiro é reavaliado a cada passo de simulação mesmo que nada tenha mudado, a abordagem reativa evita trabalho redundante e, sobretudo, aproxima a estrutura do programa da estrutura do problema: as conexões entre objetos passam a ser o próprio grafo de dependências, assim como os fios de um circuito definem quais portas reagem a quais sinais. Para o propósito didático deste trabalho, essa correspondência é a vantagem mais relevante, pois o código que o estudante escreve para conectar componentes descreve diretamente o circuito, e não um

procedimento para simulá-lo.

O **padrão de projeto *Observer*** (GAMMA et al., 1995) oferece um mecanismo direto para implementar esse comportamento em uma linguagem orientada a objetos: um objeto observado (*subject*) mantém uma lista de observadores (*observers*, chamados de *listeners* neste trabalho) e os notifica sempre que seu estado muda, sem precisar conhecer os detalhes de cada observador. Aplicado à simulação de circuitos digitais, cada componente (porta lógica, somador, registrador) atua simultaneamente como observador de suas entradas e como sujeito observado por quem estiver conectado à sua saída, formando a cadeia de propagação que reproduz o comportamento de um circuito real. A arquitetura concreta do simulador desenvolvido, construída sobre esse padrão, é apresentada no Capítulo 4, precedida por uma discussão de trabalhos relacionados no Capítulo 3.

3 Trabalhos Relacionados

Este capítulo situa o simulador proposto em relação a iniciativas existentes que buscam, de formas distintas, reduzir a barreira de entrada no aprendizado de circuitos digitais. A Seção 3.1 discute simuladores gráficos e híbridos de circuitos digitais; a Seção 3.2 discute bibliotecas e HDLs embutidas em Python; a Seção 3.3 discute iniciativas pedagógicas voltadas especificamente para HDLs tradicionais como Verilog; e a Seção 3.4 sintetiza essas iniciativas em uma comparação e identifica a lacuna que este trabalho busca preencher.

3.1 Simuladores Gráficos e Híbridos de Circuitos Digitais

O Logisim (BURCH, 2002) é um simulador educacional de circuitos digitais de código aberto, voltado a disciplinas introdutórias, em que o usuário monta circuitos arrastando e conectando portas lógicas em um editor visual, sem escrever código. Miquelini e Ferrari (2021) analisaram o Logisim resolvendo exercícios de eletrônica digital e concluíram que, apesar de limitações pontuais (principalmente na ferramenta de análise usada para *flip-flops*), suas vantagens (gratuidade, leveza e possibilidade de modificação) o tornam adequado ao ensino.

O CircuitVerse (CIRCUITVERSE, 2026) adota abordagem semelhante, mas executada no navegador: o usuário monta circuitos visualmente e pode criar bancadas de teste (*testbenches*) e validar o comportamento esperado com diagramas de tempo. Savolainen (2020) comparou Logisim e CircuitVerse em um contexto educacional, segundo critérios de facilidade de uso, eficiência e teste de circuitos, e identificou vantagens distintas em cada um: o Logisim mostrou-se mais eficiente e com mais elementos para a construção de circuitos, enquanto o CircuitVerse foi considerado mais fácil para começar e adequado ao ensino.

O DigitalJS (MATERZOK, 2019) representa uma abordagem híbrida: o estudante escreve uma descrição em Verilog/SystemVerilog, que é sintetizada pela ferramenta Yosys e visualizada como um circuito de portas interativo no navegador, permitindo observar a propagação de sinais sobre o circuito sintetizado a partir do próprio código HDL. No contexto da própria UFSC, o FPGA Emulator Web (BATISTA, 2021), desenvolvido por Eduardo Luiz Ortiz Batista, segue abordagem semelhante para VHDL: uma plataforma web que utiliza o GHDL como *backend* para emular e simular descrições VHDL, sem exigir do estudante a instalação de ferramentas de síntese tradicionais.

No âmbito profissional, ferramentas como o Proteus (LABCENTER ELECTRONICS, 2026) (Labcenter Electronics) combinam esquemáticos de circuitos com simulação de modo misto baseada em SPICE, suportando circuitos analógicos, digitais e mistos. Seu caráter

proprietário e sua complexidade, no entanto, o tornam menos adequado a um primeiro contato puramente didático com os conceitos de circuitos digitais.

Em comum, essas ferramentas compartilham uma limitação relevante para os objetivos deste trabalho: operam como ambientes gráficos fechados, em que o estudante interage por meio de uma interface visual, mas não tem acesso à lógica de simulação subjacente nem pode estendê-la de forma programática.

3.2 Bibliotecas e HDLs Embutidas em Python

Uma segunda família de iniciativas modela *hardware* diretamente em Python, adotando-o como linguagem hospedeira. O BinPy (BINPY DEVELOPERS, 2014) é uma biblioteca escrita inteiramente em Python, com componentes que vão de portas lógicas a *flip-flops*, contadores e registradores, demonstrando a viabilidade de modelar eletrônica digital de forma orientada a objetos.

O MyHDL (DECALUWE, 2004) vai além da simulação: permite descrever *hardware* usando construtos nativos de Python, como geradores, e converter automaticamente um subconjunto desse código para Verilog sintetizável, funcionando como uma HDL alternativa; a conversão para VHDL foi incorporada em versões posteriores (MYHDL, 2009). O PyRTL (CLOW et al., 2017) propõe um fluxo de projeto em nível RTL descrito em Python, tendo como objetivos centrais simplicidade, usabilidade, clareza e extensibilidade, em vez de desempenho ou otimização. Mirza, Dangwal e Sherwood (2019) relatam que o PyRTL permitiu que estudantes de segundo ano de graduação conduzissem pesquisa em arquitetura de computadores apoiando-se no conhecimento prévio de Python, em vez de exigir o aprendizado antecipado de Verilog ou VHDL, um relato de experiência, com uma equipe de quatro estudantes e análise qualitativa, que indica que a familiaridade prévia com Python pode ser aproveitada como ponte para o aprendizado de *hardware*, tal como proposto neste trabalho.

Em comum, BinPy, MyHDL e PyRTL demonstram que Python é uma linguagem hospedeira viável para modelar *hardware*, mas diferem na forma como propagam as mudanças de sinal. O PyRTL avalia o circuito ciclo a ciclo, por meio de um laço de simulação. O MyHDL possui um simulador orientado a eventos, mas, assim como os simuladores de VHDL e Verilog, concentra o agendamento desses eventos em um núcleo de simulação central, separado dos componentes descritos pelo estudante. O BinPy é o que mais se aproxima da abordagem deste trabalho: suas portas lógicas podem ser ligadas por objetos `Connector` que, ao mudar de estado, disparam o recálculo das portas conectadas a eles, uma propagação distribuída entre os componentes semelhante à descrita na Seção 2.3. Nos elementos sequenciais, porém, alterar as entradas não atualiza o componente, e é necessário chamar explicitamente o método `trigger()` (BINPY DEVELOPERS, 2014); além disso, o projeto não recebe atualizações de código desde 2016 e exige a instalação de

um pacote à parte.

No contexto local, o projeto simuGATE (SEEMANN; BOING, 2020), desenvolvido por estudantes do Programa de Educação Tutorial de Engenharia Elétrica da UFSC (PET EEL), é um simulador de portas lógicas (OR, AND, NOT, NAND, NOR, XOR) implementado em Python com interface gráfica baseada na biblioteca Kivy, criado para ajudar estudantes de disciplinas de circuitos a simular o funcionamento dessas portas de forma simples. Seus componentes são apenas combinacionais, e a saída de cada porta é calculada a partir de uma lista de valores de entrada, sem propagação automática entre portas conectadas. Diferentemente das bibliotecas anteriores, e assim como o Logisim e o CircuitVerse, o simuGATE é voltado ao uso por meio de uma interface gráfica, e não à leitura do código dos componentes; o projeto, contudo, ficou em estágio de protótipo, com a ligação entre portas ainda não implementada.

3.3 Iniciativas Pedagógicas para HDLs Tradicionais

Um terceiro grupo de trabalhos não propõe alternativas às HDLs tradicionais, mas sim formas de facilitar seu aprendizado direto. Ebeling e French (2007) propuseram o Abstract Verilog, uma linguagem semelhante ao Verilog, com sintaxe restrita e semântica de execução paralela bem definida, para reduzir a carga cognitiva de estudantes iniciantes sem deixar de representar *hardware*. Jamieson (2017) desenvolveu o verilogTown, um jogo educacional em que o jogador escreve trechos de Verilog para controlar sinais de trânsito, buscando engajar o estudante por meio de um ambiente lúdico; o autor relata que a turma que utilizou o jogo teve, em um dos exames, média cerca de 9 pontos percentuais superior à da turma do ano anterior, ressalvando que a comparação não foi acompanhada de testes estatísticos.

Imamura (2005) descreve uma abordagem baseada em FPGAs de baixo custo para ensinar projeto de sistemas digitais a estudantes de Ciência da Computação com pouca ou nenhuma formação prévia em Engenharia Elétrica, argumentando que esse conteúdo é tradicionalmente associado a currículos de Engenharia. Em linha semelhante, Azevedo (2017) propôs, para o ensino de Eletrônica Digital, um método baseado em FPGA e VHDL fundamentado na aprendizagem ativa e o comparou à montagem tradicional de circuitos em protoboard; embora conclua que o FPGA pode ser usado como ferramenta de ensino, os estudantes consultados demonstraram resistência ao método: dos 15 respondentes, 46,67% preferiram a montagem tradicional, outros 46,67% preferiram ambos os métodos e nenhum optou exclusivamente pela descrição de circuitos em VHDL e FPGA, resultado que reforça a barreira de entrada das HDLs discutida no Capítulo 1. Esses trabalhos evidenciam que a dificuldade de aprendizado discutida na motivação deste trabalho (Capítulo 1) é reconhecida na literatura, mas suas soluções propostas (uma HDL simplificada, um jogo educacional, *hardware* reconfigurável físico) atuam depois que o estudante já está diante

da HDL ou do *hardware* real, e não como uma etapa preparatória anterior a esse contato.

3.4 Síntese Comparativa e Lacuna Identificada

A Tabela 2 compara as ferramentas discutidas neste capítulo e o simulador proposto segundo oito critérios. Alguns favorecem este trabalho e outros favorecem as demais ferramentas; a intenção é situar o simulador de forma justa, e não apenas destacar suas vantagens. Os critérios são:

- **Interface**: se o circuito é montado por interface gráfica (G), por código (C) ou por código HDL visualizado graficamente (H);
- **Sem instalação**: se a ferramenta pode ser usada no navegador ou com uma instalação padrão de Python, sem pacotes ou programas adicionais;
- **Sequenciais (Seq.)**: se a ferramenta simula circuitos sequenciais (*latches*, *flip-flops*, registradores);
- **Diagrama de tempo**: se oferece visualização da evolução dos sinais ao longo do tempo (diagrama de tempo ou forma de onda);
- **Atraso**: se modela atraso de propagação das portas;
- **HDL/FPGA**: se gera ou simula código HDL sintetizável, permitindo chegar a um circuito real em FPGA;
- **Mecanismo no código**: se a propagação de sinais é implementada pelos próprios componentes que o estudante instancia, em código que ele lê e pode modificar, como no padrão *Observer* (Seção 2.3), em vez de ser conduzida por uma interface gráfica, por chamadas explícitas ou por um núcleo de simulação separado;
- **Aberto**: se o código-fonte é aberto.

Tabela 2 – Comparação entre trabalhos relacionados e o simulador proposto.

Ferramenta	**Plataforma**	**Interf.**	**Sem instal.**	**Seq.**	**Diagr. tempo**	**Atraso**	**HDL/ FPGA**	**Mecan. no cód.**	**Aberto**
Logisim	Java	G	✗	✓	✗[a]	✓	✗[a]	✗	✓
CircuitVerse	Navegador	G	✓	✓	✓	✓	✓	✗	✓
DigitalJS	Navegador (Verilog)	H	✓	✓	✓	?	✓	✗	✓
FPGA Emulator Web	Navegador (VHDL)	H	✓	✓	?	?	✓	✗	✓
Proteus	Windows	G	✗	✓	✓	✓	✗	✗	✗
BinPy	Python	C	P[b]	✓	P[c]	✗	✗	P[d]	✓
MyHDL	Python	C	P[b]	✓	✓	✓	✓	P[e]	✓
PyRTL	Python	C	P[b]	✓	✓	✗	✓	P[e]	✓
simuGATE	Python/Kivy	G	✗	✗	✗	✗	✗	✗	P[f]
Este trabalho	**Python**	C	✓	✓	✗	✗	✗	✓	✓

✓: atende; ✗: não atende; P: atende parcialmente; ?: não foi possível verificar na documentação disponível. Seq.: circuitos sequenciais.
[a] O Logisim-evolution, derivado do Logisim, oferece cronograma de sinais e exportação para VHDL.
[b] Exige apenas instalar o pacote com o gerenciador `pip`.
[c] Osciloscópio em modo texto, escrito para Python 2.
[d] Propagação distribuída por objetos `Connector` nas portas lógicas; nos elementos sequenciais, a atualização exige chamada explícita ao método `trigger()`.
[e] O código é Python aberto, mas a propagação é conduzida por um núcleo de simulação (MyHDL) ou por um laço de simulação por ciclos (PyRTL), separado dos componentes.
[f] Código-fonte público, mas sem licença de uso declarada no repositório.

Fonte: elaborada pelo autor (2026), com base na documentação das ferramentas.

A comparação mostra, em primeiro lugar, que este trabalho não atende a três critérios atendidos por várias das outras ferramentas: não oferece diagrama de tempo, não modela atraso de propagação e não gera código HDL nem permite chegar a um circuito em FPGA. Também não oferece interface gráfica para montagem de circuitos. Essas ausências são coerentes com o propósito do trabalho, que é servir como etapa preparatória para o estudo de circuitos digitais, e não substituir as ferramentas existentes:

- a ausência de **interface gráfica** é deliberada: o objetivo é que o estudante descreva o circuito em texto, aproximando-se da forma de trabalho das HDLs que encontrará em seguida, papel que as ferramentas gráficas, já bem estabelecidas, não cumprem;
- o **diagrama de tempo** é substituído, no escopo atual, pela impressão do estado a cada ciclo e pelos recursos de observação da Seção 5.3; sua implementação é possível sobre o mesmo mecanismo de *listeners* e é apontada como trabalho futuro;
- a ausência de **atraso de propagação** é uma simplificação: os conteúdos introdutórios visados (portas lógicas, somadores, registradores e contadores) dependem do comportamento lógico dos circuitos, e não de sua temporização; essa limitação é discutida na Seção 5.4;

- a **geração de HDL e síntese em FPGA** estão fora do escopo por definição: o simulador não pretende ser uma HDL, mas preparar o estudante para aprendê-la. Para esse fim, MyHDL e PyRTL, bem como as próprias HDLs, são as ferramentas adequadas.

Por outro lado, a comparação evidencia a lacuna que motiva este trabalho: nenhuma das ferramentas revisadas combina simultaneamente (i) uma linguagem já familiar ao estudante de Ciência da Computação, (ii) uso sem instalação além de um ambiente Python padrão e (iii) um mecanismo de propagação implementado pelos próprios componentes, em código que o estudante lê e pode modificar. As ferramentas gráficas e híbridas oferecem recursos mais completos de visualização e temporização, mas escondem a lógica de simulação atrás de uma interface; MyHDL e PyRTL atendem a (i), são as mais completas do ponto de vista de projeto de *hardware* e atendem parcialmente a (iii), mas conduzem a propagação por um núcleo ou laço de simulação separado dos componentes; o BinPy é o que mais se aproxima, atendendo a (i) e, nas portas lógicas, a (iii), mas exige a instalação de um pacote, depende de chamadas explícitas nos elementos sequenciais e não é mantido desde 2016; e as iniciativas pedagógicas para HDLs tradicionais (Abstract Verilog, verilogTown) atuam sobre a própria HDL, sem oferecer uma etapa preparatória em uma linguagem já conhecida. É essa combinação específica, detalhada na Seção 2.3 e implementada no Capítulo 4, que diferencia o simulador proposto neste trabalho.

4 Proposta

Este capítulo apresenta a proposta deste trabalho e descreve sua realização: uma biblioteca em Python que simula circuitos digitais por meio de propagação reativa de sinais, situada em relação aos trabalhos relacionados discutidos no Capítulo 3.

4.1 Visão Geral da Proposta

A proposta consiste em uma biblioteca de simulação de circuitos digitais, e não em uma ferramenta gráfica ou uma linguagem de descrição de *hardware*: o estudante escreve Python comum, instanciando e conectando objetos que representam sinais, portas lógicas e componentes sequenciais, e observa o comportamento do circuito diretamente no interpretador ou em um *script*. Não há passo de compilação/síntese separado, nem ambiente de desenvolvimento dedicado: qualquer instalação padrão de Python 3, sem dependências externas, é suficiente.

Tecnicamente, a proposta se resume a dois compromissos de projeto:

- **propagação reativa**, e não avaliação sob demanda: cada componente recalcula e notifica automaticamente sua saída sempre que uma de suas entradas muda (padrão *Observer*, Seção 2.3), em vez de exigir uma chamada explícita de "simular" ou "avaliar" o circuito;
- **composição a partir de portas lógicas** como forma preferencial de construir componentes maiores (somadores, *latches*, registradores), reservando descrições comportamentais em lógica Python (Seção 2.2; como o `BehavioralAdder`, Seção 4.6, e o `BehavioralRegister`, Seção 4.7) para quando o objetivo de um teste ou demonstração é outra parte do circuito.

O restante deste capítulo descreve a implementação que realiza essa proposta: a arquitetura geral (Seção 4.2), a representação de sinais (Seção 4.3), as portas lógicas (Seção 4.4), os dois somadores (Seções 4.5 e 4.6), os componentes sequenciais, incluindo o registrador em lógica Python e sua contraparte construída com *latches* (Seções 4.7 e 4.8), e a composição desses componentes nos circuitos de exemplo validados no Capítulo 5. Ao final (Seção 4.11), a *engine* de propagação é revisitada em conjunto, amarrando o que cada seção apresentou separadamente.

4.2 Visão Geral da Arquitetura

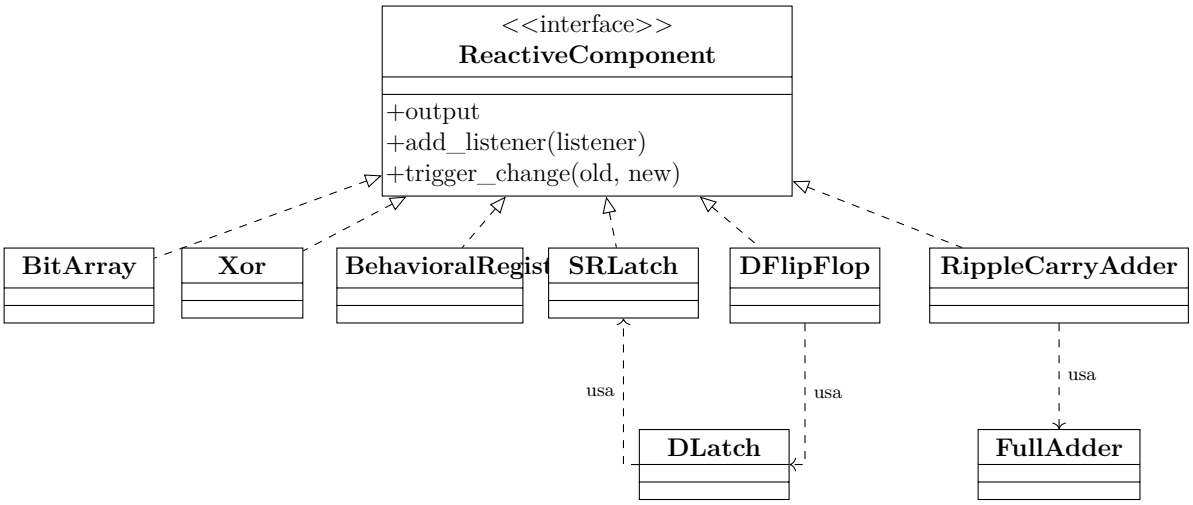
Todos os componentes do simulador seguem o mesmo contrato, definido informalmente pelas seguintes três operações:

- uma propriedade `output`, que expõe o valor atual do sinal do componente (um valor booleano ou uma lista de valores booleanos);
- um método `add_listener(fn)`, que registra uma função `fn(old, new)` a ser chamada sempre que `output` mudar;
- um método `trigger_change(old, new)`, chamado internamente para recalcular a saída do componente e notificar seus próprios *listeners*.

Esse contrato implementa o padrão *Observer* descrito na Seção 2.3: cada componente é, ao mesmo tempo, observador das suas entradas (registrando-se como *listener* delas no momento em que é construído) e sujeito observado por quem estiver conectado à sua saída.

O contrato não é implementado por meio de uma interface ou classe abstrata explícita em Python (nenhuma classe deste trabalho herda de outra): cada componente simplesmente define os mesmos três membros por conta própria, e o encaixe entre eles é garantido pela convenção, não pelo compilador. A Figura 2 representa esse contrato implícito como a interface `ReactiveComponent`, e mostra, para um subconjunto representativo de classes de cada pacote, a relação de realização (linha tracejada com seta vazada) e as principais dependências de composição entre componentes (linha tracejada rotulada "usa").

Figura 2 – Diagrama de classes (subconjunto representativo) do contrato reativo comum.

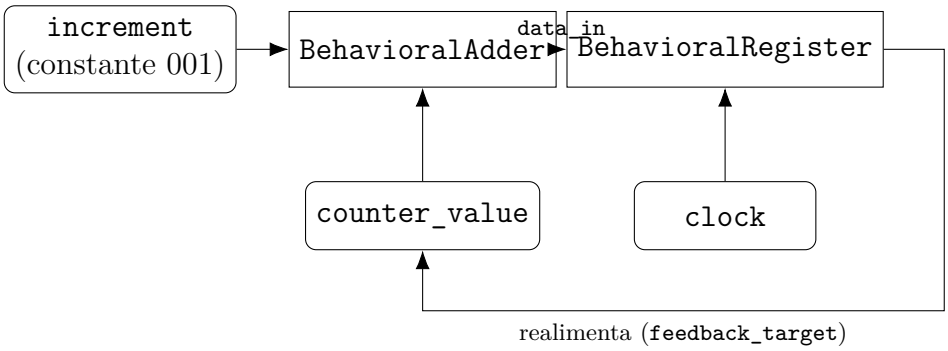


Fonte: elaborada pelo autor (2026).

A Figura 3 ilustra essa cadeia de observadores para o circuito contador que será detalhado na Seção 4.9: cada seta indica um componente registrado como *listener* de

outro, e o laço inferior mostra o valor capturado pelo registrador sendo realimentado para a entrada do somador.

Figura 3 – Cadeia de propagação reativa do contador binário realimentado.



Fonte: elaborada pelo autor (2026).

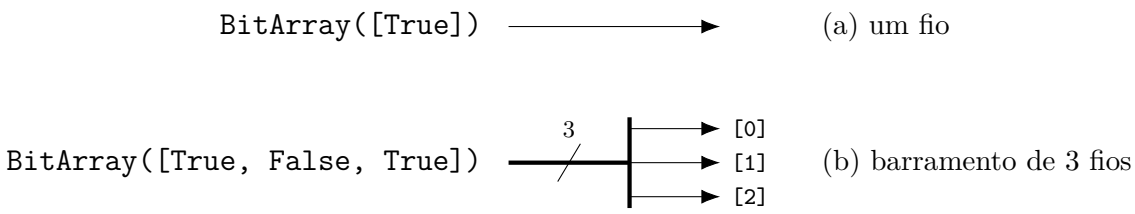
4.3 Representação de Sinais

O pacote `signals` contém as três classes responsáveis por representar sinais/barramentos de *bits*.

4.3.1 `BitArray`

A classe `BitArray` representa um sinal, seja ele um *bit* único ou um barramento de múltiplos *bits*. Em termos de circuito, um `BitArray` de um único *bit* corresponde a um fio, e um `BitArray` de N *bits* corresponde a um conjunto de N fios agrupados em um barramento, como ilustra a Figura 4.

Figura 4 – Um fio (`BitArray` de 1 *bit*) e um barramento de três fios (`BitArray` de 3 *bits*).



Fonte: elaborada pelo autor (2026).

Seu construtor recebe o valor inicial, e a propriedade `output` intercepta toda atribuição para notificar os *listeners* registrados, como mostra o Algoritmo 1.

Algoritmo 1 – Classe BitArray (signals/bit_array.py).

```
1  class BitArray:
2      def __init__(self, initial_value):
3          self._output = initial_value
4          self._listeners = []
5
6      @property
7      def output(self):
8          return self._output
9
10     @output.setter
11     def output(self, value):
12         previous_output = self._output
13         if value == previous_output:
14             return
15         self._output = value
16         for listener in self._listeners:
17             listener(previous_output, value)
18
19     def add_listener(self, listener):
20         self._listeners.append(listener)
```

Fonte: elaborado pelo autor (2026).

O uso do decorador `@property` (Linhas 6 e 10 do Algoritmo 1) é o que torna a notificação transparente: do ponto de vista do código cliente, `sinal.output = novo_valor` é uma simples atribuição, mas internamente ela dispara a notificação de todos os componentes conectados a esse sinal.

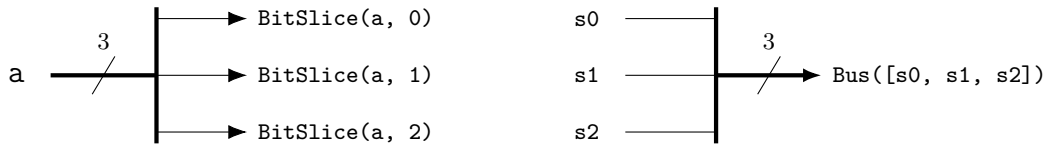
4.3.2 `BitSlice` e `Bus`

As portas lógicas (Seção 4.4) operam elemento a elemento sobre barramentos do mesmo tamanho, mas não propagam *carry* entre posições de *bit*. Para montar um somador de N *bits* com propagação de *carry* (Seção 4.5), é preciso operar em cada *bit* individualmente. Duas classes auxiliares resolvem esse problema:

- `BitSlice(source, index)` projeta um único *bit* de um sinal maior como um sinal próprio de 1 *bit*, atualizando-se automaticamente quando `source` muda;
- `Bus(bits)` faz o caminho inverso, combinando uma lista de sinais de 1 *bit* (do mais ao menos significativo) em um único sinal de N *bits*.

Em termos de fios, `BitSlice` corresponde a retirar um único fio de um barramento, e `Bus` a agrupar vários fios independentes em um novo barramento, como ilustra a Figura 5.

Figura 5 – `BitSlice` retirando fios de um barramento e `Bus` agrupando fios em um barramento.



(a) `BitSlice` (b) `Bus`

Fonte: elaborada pelo autor (2026).

Ambas seguem o mesmo contrato (`output`, `add_listener`, `trigger_change`) e são usadas internamente pelo `RippleCarryAdder`, descrito na Seção 4.5.

4.4 Portas Lógicas

As portas lógicas (`Not`, `And`, `Or`, `Xor`, no pacote `gates`) seguem todas a mesma estrutura: recebem seus componentes de entrada no construtor, se registram como *listeners* deles e calculam a saída *bit* a *bit*. O Algoritmo 2 apresenta, como exemplo, a porta `Xor`.

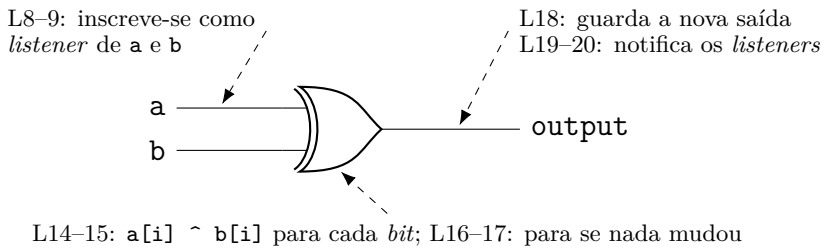
Algoritmo 2 – Porta Xor (gates/xor_gate.py).

```
1  class Xor:
2      def __init__(self, a, b):
3          self._a = a
4          self._b = b
5          self._output = None
6          self._listeners = []
7
8          self._a.add_listener(self.trigger_change)
9          self._b.add_listener(self.trigger_change)
10         self.trigger_change(None, None)
11
12     def trigger_change(self, old, new):
13         previous_output = self._output
14         new_value = [self._a.output[i] ^ self._b.output[i]
15                      for i in range(len(self._a.output))]
16         if new_value == previous_output:
17             return self._output
18         self._output = new_value
19         for listener in self._listeners:
20             listener(previous_output, new_value)
21         return self._output
22
23     @property
24     def output(self):
25         return self._output
26
27     def add_listener(self, listener):
28         self._listeners.append(listener)
```

Fonte: elaborado pelo autor (2026).

As demais portas (`Not`, `And`, `Or`) diferem apenas na operação booleana aplicada a cada par de *bits* das entradas (Linha 14 do Algoritmo 2). Como cada porta se inscreve como *listener* de suas entradas já no construtor (Linhas 8 e 9 do Algoritmo 2), a saída de uma porta é recalculada automaticamente sempre que qualquer uma de suas entradas muda, sem que o código cliente precise chamar nenhum método explicitamente. A Figura 6 relaciona cada parte do símbolo da porta às linhas do Algoritmo 2 que a implementam.

Figura 6 – Porta XOR anotada com as linhas do Algoritmo 2.



Fonte: elaborada pelo autor (2026).

4.5 Somador com Portas Lógicas

Esta é a forma recomendada de montar um somador no simulador: inteiramente a partir das portas lógicas básicas, sem nenhum atalho em Python. Um somador completo de 1 *bit* é montado com dois meios-somadores mais uma porta OR para combinar os dois possíveis *carries*, como mostra o Algoritmo 3.

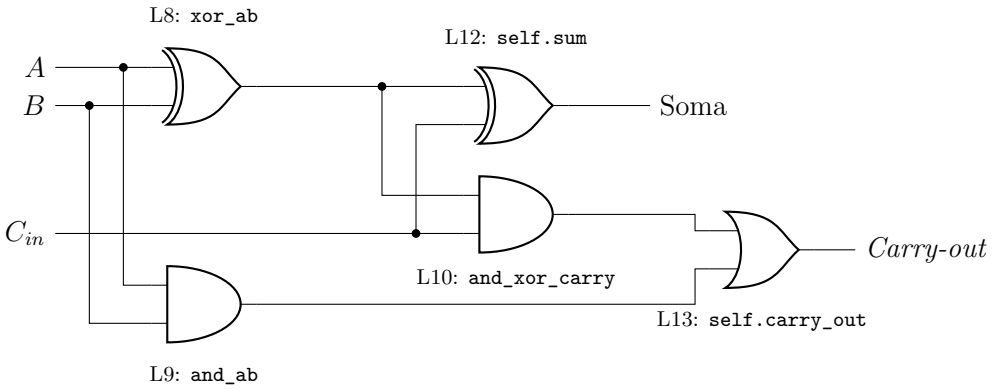
Algoritmo 3 – FullAdder (components/full_adder.py).

```
1  from gates.xor_gate import Xor
2  from gates.and_gate import And
3  from gates.or_gate import Or
4
5
6  class FullAdder:
7      def __init__(self, a, b, carry_in):
8          xor_ab = Xor(a, b)
9          and_ab = And(a, b)
10         and_xor_carry = And(xor_ab, carry_in)
11
12         self.sum = Xor(xor_ab, carry_in)
13         self.carry_out = Or(and_ab, and_xor_carry)
```

Fonte: elaborado pelo autor (2026).

A Figura 7 mostra o mesmo circuito representado com os símbolos de portas lógicas apresentados na Seção 2.1, evidenciando a correspondência direta entre o diagrama e o código: cada porta lógica do diagrama corresponde a uma instância de `Xor`, `And` ou `Or` no construtor de `FullAdder`, e está anotada com a linha do Algoritmo 3 que a cria.

Figura 7 – Somador completo (*full adder*) montado com portas lógicas, com a linha do Algoritmo 3 correspondente a cada porta.



Fonte: elaborada pelo autor (2026).

`FullAdder` expõe `.sum` e `.carry_out` como saídas de portas lógicas comuns (Linhas 12 e 13 do Algoritmo 3), prontas para servir de entrada a outro componente.

O `RippleCarryAdder`, apresentado no Algoritmo 4, encadeia N instâncias de `FullAdder`, uma por *bit*, do menos significativo ao mais significativo, usando `BitSlice` para extrair cada *bit* dos barramentos de entrada e um `Bus` para recompor os N *bits* de soma em um único sinal.

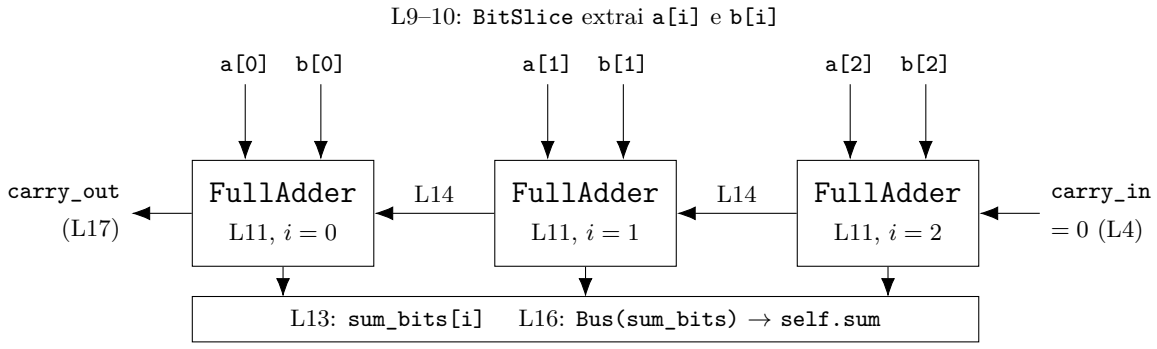
Algoritmo 4 – RippleCarryAdder (components/ripple_carry_adder.py).

```
1  class RippleCarryAdder:
2      def __init__(self, a, b, carry_in=None):
3          width = len(a.output)
4          carry = carry_in if carry_in is not None else BitArray([False])
5
6          sum_bits = [None] * width
7          self.full_adders = []
8          for i in range(width - 1, -1, -1):
9              bit_a = BitSlice(a, i)
10             bit_b = BitSlice(b, i)
11             full_adder = FullAdder(bit_a, bit_b, carry)
12             self.full_adders.append(full_adder)
13             sum_bits[i] = full_adder.sum
14             carry = full_adder.carry_out
15
16         self.sum = Bus(sum_bits)
17         self.carry_out = carry
```

Fonte: elaborado pelo autor (2026).

A Figura 8 mostra a estrutura resultante para N = 3, anotada com as linhas do código: o laço das Linhas 8 a 14 do Algoritmo 4 percorre os *bits* do menos significativo (índice 2, à direita) ao mais significativo (índice 0, à esquerda), e a variável `carry` (Linha 14) conecta o *carry-out* de cada `FullAdder` ao *carry-in* do seguinte. A lista `full_adders` (Linhas 7 e 12) não participa da soma: ela apenas guarda os estágios criados, para que possam ser inspecionados depois da construção (Seção 5.3).

Figura 8 – `RippleCarryAdder` de 3 *bits* formado pelo encadeamento de três `FullAdder`, com as linhas do Algoritmo 4.



Fonte: elaborada pelo autor (2026).

Um ponto central deste trabalho é que, aqui, o *overflow* não é tratado por nenhuma função de *padding* ou truncamento: ele surge naturalmente da topologia do circuito. A soma só tem N fios de saída, um por estágio de `FullAdder`, e o "vai-um" do último estágio vira um sinal `carry_out` separado. Quando esse sinal não é conectado a mais nada, a informação do estouro se perde e a soma "dá a volta" sozinha, exatamente como aconteceria em um contador de *hardware* real. Esse comportamento é validado e discutido em detalhe na Seção 5.1.5.

4.6 `BehavioralAdder`: um Somador Comportamental em Python

O pacote `components` também inclui o `BehavioralAdder`, apresentado no Algoritmo 5, que soma dois `BitArray` sem montar nenhuma porta lógica: ele segue o mesmo contrato reativo das portas, mas, em vez de uma operação booleana, delega o cálculo da saída (Linha 14) à função `sum_via_integer_conversion`.

Algoritmo 5 – BehavioralAdder (components/behavioral_adder.py).

```
1  class BehavioralAdder:
2      def __init__(self, a, b):
3          self._a = a
4          self._b = b
5          self._output = None
6          self._listeners = []
7
8          self._a.add_listener(self.trigger_change)
9          self._b.add_listener(self.trigger_change)
10         self.trigger_change(None, None)
11
12     def trigger_change(self, old, new):
13         previous_output = self._output
14         new_value = sum_via_integer_conversion(self._a.output, self._b.output)
15         if new_value == previous_output:
16             return self._output
17         self._output = new_value
18         for listener in self._listeners:
19             listener(previous_output, new_value)
20         return self._output
21
22     @property
23     def output(self):
24         return self._output
25
26     def add_listener(self, listener):
27         self._listeners.append(listener)
```

Fonte: elaborado pelo autor (2026).

As funções auxiliares usadas pelo `BehavioralAdder` estão no módulo `utils/binary_conversion.p` apresentado no Algoritmo 6: `sum_via_integer_conversion` (Linhas 1–6) converte as

duas entradas para inteiro com `binary_array_to_int` (Linhas 9–11), soma os inteiros em Python, converte o resultado de volta para uma lista de *bits* com `int_to_binary_array` (Linhas 14–16) e, por fim, ajusta essa lista ao tamanho da entrada com `pad_binary_array` (Linhas 19–24).

Algoritmo 6 – Funções auxiliares de conversão e *padding* binário (utils/binary_conversion.py).

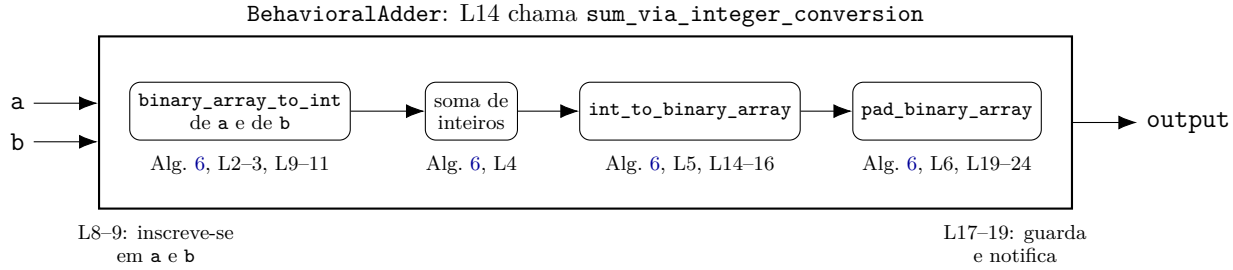
```
1  def sum_via_integer_conversion(a, b):
2      a_int = binary_array_to_int(a)
3      b_int = binary_array_to_int(b)
4      sum_int = a_int + b_int
5      sum_array = int_to_binary_array(sum_int)
6      return pad_binary_array(sum_array, len(a))
7
8
9  def binary_array_to_int(binary_array):
10     bits_as_str = "".join("1" if bit else "0" for bit in binary_array)
11     return int(bits_as_str, 2)
12
13
14 def int_to_binary_array(value):
15     binary_str = bin(value)[2:]
16     return [bit == "1" for bit in binary_str]
17
18
19 def pad_binary_array(binary_array, length):
20     if len(binary_array) < length:
21         return [False] * (length - len(binary_array)) + binary_array
22     if len(binary_array) > length:
23         return binary_array[-length:]
24     return binary_array[:]
```

Fonte: elaborado pelo autor (2026).

Como o nome indica, trata-se de uma **descrição comportamental** do somador, no mesmo sentido usado nas HDLs (Seção 2.2): o componente descreve o que o somador faz, e não de quais portas ele é feito, como ocorre na descrição estrutural do `RippleCarryAdder`. Não se trata, portanto, de uma tentativa imperfeita de somador digital, e sim de uma abstração deliberada: um recurso do simulador que demonstra a possibilidade de **misturar um trecho de Python "puro" com o restante do circuito reativo**. Isso é útil quando o objetivo de um teste ou demonstração é outra parte do circuito, por exemplo, o comportamento do registrador e da realimentação de um contador, e não a aritmética em si. Nesses casos, o `BehavioralAdder` permite ao estudante montar rapidamente um circuito funcional sem precisar implementar previamente um somador com portas lógicas, substituindo-o pelo `RippleCarryAdder` (Seção 4.5) somente quando o próprio somador for o objeto de estudo.

A Figura 9 representa o `BehavioralAdder` como um bloco: externamente, ele tem as mesmas entradas e a mesma saída de um somador de portas lógicas; internamente, em vez de portas interligadas (como na Figura 8), há apenas uma sequência de operações em Python, cada uma anotada com as linhas dos Algoritmos 5 e 6 que a implementam.

Figura 9 – `BehavioralAdder` como bloco comportamental, com as linhas dos Algoritmos 5 e 6.



Fonte: elaborada pelo autor (2026).

A função `pad_binary_array` merece destaque: ela garante que o resultado da soma tenha exatamente o mesmo número de *bits* da entrada original, preenchendo com zeros à esquerda quando o resultado é menor (Linhas 20 e 21 do Algoritmo 6), e **truncando os *bits* mais significativos** quando o resultado excede esse tamanho (Linhas 22 e 23), o que reproduz o *overflow* apenas dentro da descrição comportamental do `BehavioralAdder`. É importante notar que essa função é usada exclusivamente por `sum_via_integer_conversion`: quando o somador é montado com portas lógicas (`RippleCarryAdder`), como discutido na Seção 4.5, o *overflow* é tratado de forma completamente diferente, pelo sinal `carry_out`, e `pad_binary_array` não entra em jogo.

O truncamento das Linhas 22 e 23 não existia na primeira versão da função, que apenas preenchia o resultado com zeros à esquerda. Ao executar o contador de 3 *bits* com `BehavioralAdder` além do valor `111`, o oitavo pulso de *clock* produzia `[True, False, False, False]`: em vez de voltar a `000`, como um contador real, o contador crescia em largura. O episódio ilustra que a propagação automática de valores garante que os componentes sejam recalculados na ordem correta, mas não que cada componente reproduza fielmente o comportamento do *hardware*; essa fidelidade precisa ser verificada componente a componente, sobretudo nas descrições comportamentais, em que a lógica é escrita livremente em Python.

4.7 Componente Sequencial: `BehavioralRegister`

O `BehavioralRegister` é o componente do simulador com comportamento sensível a borda de *clock* mais simples de usar, e é o que dá início aos circuitos sequenciais: qualquer circuito com estado (contador, *flip-flop*) depende de um registrador para "lembrar" um

valor entre pulsos de *clock*. É importante ser explícito sobre sua natureza, que o próprio nome indica, assim como no `BehavioralAdder`: o `BehavioralRegister` é uma **descrição comportamental em lógica Python**, não um circuito montado a partir de portas lógicas; sua contraparte estrutural é o `DFlipFlopRegister` (Seção 4.8.3). Ele não modela o funcionamento interno de um *flip-flop* real (não há *latches*, não há a estrutura mestre-escravo discutida na Seção 4.8); ele apenas verifica, em Python, se o *clock* transicionou de falso para verdadeiro e, em caso afirmativo, lê diretamente o valor atual de sua entrada. Ele recebe um sinal `clock`, um componente `data_in` (o dado a ser capturado) e, opcionalmente, um componente `feedback_target` a ser notificado quando o registrador atualizar seu valor, como mostra o Algoritmo 7.

Algoritmo 7 – Classe BehavioralRegister (components/behavioral_register.py).

```
1  def _is_high(value):
2      if isinstance(value, list):
3          return bool(value) and bool(value[0])
4      return bool(value)
5
6
7  class BehavioralRegister:
8      def __init__(self, clock, data_in, feedback_target=None):
9          self._clock = clock
10         self._data_in = data_in
11         self._output = None
12         self._listeners = []
13
14         self._clock.add_listener(self.trigger_change)
15         if feedback_target is not None:
16             self.add_listener(feedback_target.trigger_change)
17
18     def trigger_change(self, old, new):
19         previous_output = self._output
20         if not _is_high(old) and _is_high(new):
21             self._output = self._data_in.output
22             for listener in self._listeners:
23                 listener(previous_output, self._output)
24         return self._output
25
26     @property
27     def output(self):
28         return self._output
29
30     def add_listener(self, listener):
31         self._listeners.append(listener)
```

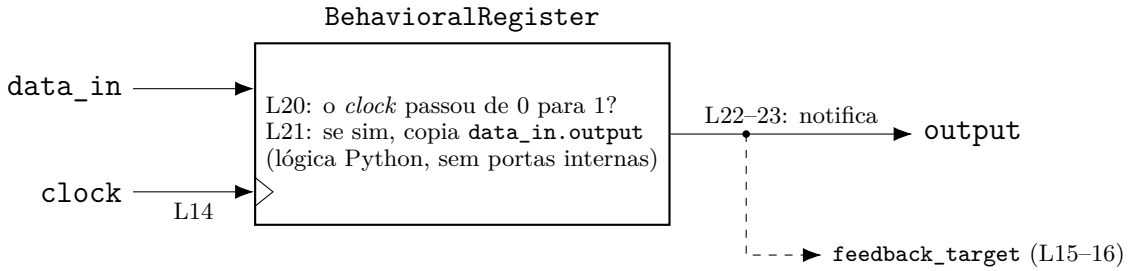
Fonte: elaborado pelo autor (2026).

O condicional da Linha 20 do Algoritmo 7 é o que implementa a detecção de borda de subida: o valor de `data_in.output` só é capturado quando `clock.output` muda de falso para verdadeiro. A função auxiliar `_is_high` (Linhas 1–4) existe porque o *clock* pode

ser representado tanto por um booleano (`BitArray(False)`) quanto por uma lista de um único *bit* (`BitArray([False])`), forma exigida quando o mesmo sinal também alimenta portas lógicas (Seção 4.9.5). Em Python, qualquer lista não vazia é considerada verdadeira, inclusive `[False]`; uma versão anterior deste componente testava apenas a "veracidade" do novo valor e, com um *clock* em forma de lista, capturava o dado também na borda de descida. Esse defeito foi identificado durante a revisão do trabalho e passou a ser coberto por um teste automatizado (Seção 5.2). Para gerar um pulso de *clock*, é necessário alternar explicitamente o valor do *clock* entre falso e verdadeiro (`clock.output = False` seguido de `clock.output = True`); atribuir verdadeiro repetidamente, sem antes voltar a falso, não gera uma nova borda e, portanto, não atualiza o registrador.

O parâmetro opcional `feedback_target` (Linhas 15 e 16) é o que permite fechar um laço de realimentação: ao passar o próprio sinal de entrada de um circuito como `feedback_target` do registrador, o valor atualizado do registrador é propagado de volta para esse sinal, fazendo com que o circuito combinacional ligado a ele seja recalculado antes do próximo pulso de *clock*. Esse mecanismo é a base dos circuitos com estado apresentados a seguir. A Figura 10 representa o `BehavioralRegister` com o símbolo usual de um registrador e indica as linhas do Algoritmo 7 responsáveis por cada parte; no lugar dos *latches* internos de um registrador real (Seção 4.8), há apenas um condicional em Python.

Figura 10 – `BehavioralRegister` representado como registrador, com as linhas do Algoritmo 7.



Fonte: elaborada pelo autor (2026).

4.8 Registrador Real com *Latches*

Como discutido na Seção 4.7, o `BehavioralRegister` é uma abstração. Para complementá-lo com uma implementação que reproduz de fato a estrutura interna de um *flip-flop*, o pacote `components` também inclui um registrador construído inteiramente a partir de portas lógicas, em três camadas: um ***latch* SR**, um ***latch* D** e um ***flip-flop* D mestre-escravo**.

4.8.1 *Latch* SR

O *latch* SR é o elemento de memória mais simples: duas portas NOR cruzadas, em que a saída de cada uma alimenta a entrada da outra. Essa realimentação combinacional genuína (a saída Q depende de $\overline{Q}$, que depende de Q) não pode ser expressa com a ordem de construção usual dos demais componentes deste trabalho, em que cada objeto recebe suas entradas já prontas no construtor: aqui, no momento em que a primeira porta NOR precisa ser criada, a segunda ainda não existe. A solução adotada, apresentada no Algoritmo 8, usa um `BitArray` comum como sinal provisório (um "proxy"), montado antes das duas portas NOR (Linha 7) e sincronizado ao valor real assim que ambas existem (Linhas 12 e 13).

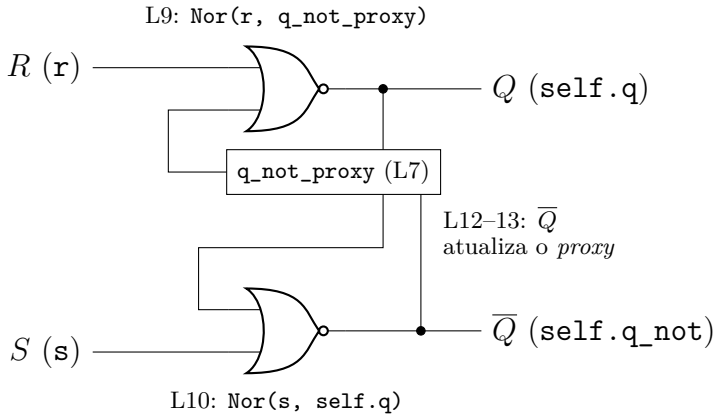
Algoritmo 8 – SRLatch (components/sr_latch.py).

```
1  from signals.bit_array import BitArray
2  from gates.nor_gate import Nor
3
4
5  class SRLatch:
6      def __init__(self, s, r):
7          q_not_proxy = BitArray([False])
8
9          self.q = Nor(r, q_not_proxy)
10         self.q_not = Nor(s, self.q)
11
12         self.q_not.add_listener(q_not_proxy.trigger_change)
13         q_not_proxy.output = self.q_not.output
```

Fonte: elaborado pelo autor (2026).

A Figura 11 mostra o mesmo circuito com o símbolo das duas portas NOR: a porta superior corresponde à Linha 9 do Algoritmo 8, a inferior à Linha 10, e o fio de realimentação de $\overline{Q}$ para a porta superior passa pelo *proxy* criado na Linha 7 e sincronizado nas Linhas 12 e 13.

Figura 11 – *Latch* SR de portas NOR cruzadas, com as linhas do Algoritmo 8.



Fonte: elaborada pelo autor (2026).

Esse laço de realimentação só funciona porque as portas lógicas do simulador (Seção 4.4) foram ajustadas para notificar seus *listeners* apenas quando o valor calculado realmente muda; sem essa condição, a notificação de uma mudança em Q dispararia uma nova notificação em $\overline{Q}$, que dispararia outra em Q, indefinidamente. A Seção 5.1.6 discute o que acontece quando essa condição de parada, sozinha, não é suficiente.

4.8.2 *Latch* D e *Flip-Flop* D Mestre-Escravo

Um *latch* D elimina a combinação inválida do *latch* SR ($S = R = 1$) derivando S e R de um único sinal de dado D e de um sinal de habilitação (*enable*): $S = D \wedge \mathit{enable}$ e $R = \overline{D} \wedge \mathit{enable}$, de modo que S e R nunca são verdadeiros ao mesmo tempo. Enquanto *enable* está em nível alto, o *latch* é transparente e acompanha D; quando *enable* cai, ele retém o último valor observado. O Algoritmo 9 implementa essas expressões nas Linhas 8 a 10.

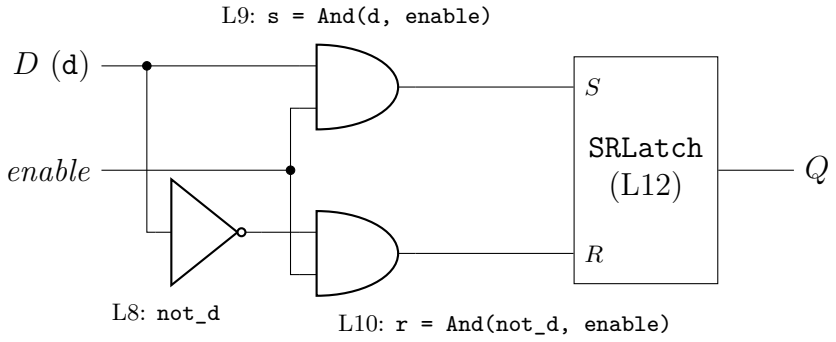
Algoritmo 9 – DLatch (components/d_latch.py).

```
1  from gates.and_gate import And
2  from gates.not_gate import Not
3  from components.sr_latch import SRLatch
4
5
6  class DLatch:
7      def __init__(self, d, enable):
8          not_d = Not(d)
9          s = And(d, enable)
10         r = And(not_d, enable)
11
12         self._sr_latch = SRLatch(s, r)
13
14     @property
15     def output(self):
16         return self._sr_latch.output
17
18     def add_listener(self, listener):
19         self._sr_latch.add_listener(listener)
```

Fonte: elaborado pelo autor (2026).

A Figura 12 mostra a estrutura correspondente: a porta NOT e as duas portas AND, criadas nas Linhas 8 a 10 do Algoritmo 9, geram S e R, que alimentam o *latch* SR criado na Linha 12.

Figura 12 – *Latch* D montado sobre um *latch* SR, com as linhas do Algoritmo 9.



Fonte: elaborada pelo autor (2026).

O `DFlipFlop`, apresentado no Algoritmo 10, encadeia dois *latches* D em uma estrutura mestre-escravo clássica: o *latch* mestre fica transparente enquanto o *clock* está em nível baixo ($enable = \overline{clock}$, Linha 9), acompanhando D; o *latch* escravo fica transparente enquanto o *clock* está em nível alto ($enable = clock$, Linha 10), repassando o valor já congelado pelo mestre. O efeito resultante é que a saída só muda no instante da borda de subida do *clock*, reproduzindo o mesmo comportamento observável do `BehavioralRegister`, mas a partir de portas lógicas reais.

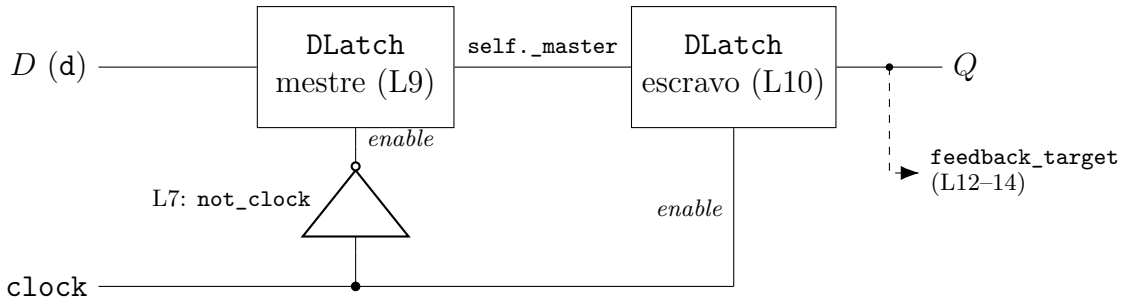
Algoritmo 10 – DFlipFlop (components/d_flip_flop.py).

```
1  from gates.not_gate import Not
2  from components.d_latch import DLatch
3
4
5  class DFlipFlop:
6      def __init__(self, d, clock, feedback_target=None):
7          not_clock = Not(clock)
8
9          self._master = DLatch(d, not_clock)
10         self._slave = DLatch(self._master, clock)
11
12         if feedback_target is not None:
13             self._slave.add_listener(feedback_target.trigger_change)
14             feedback_target.trigger_change(feedback_target.output,
               ↪  self._slave.output)
15
16     @property
17     def output(self):
18         return self._slave.output
19
20     def add_listener(self, listener):
21         self._slave.add_listener(listener)
```

Fonte: elaborado pelo autor (2026).

A sincronização explícita de `feedback_target` nas Linhas 12 a 14 do Algoritmo 10 existe pelo mesmo motivo que a do `SRLatch`: o estado interno do *flip-flop* já se estabiliza durante a própria construção (antes de qualquer pulso de *clock*), e sem essa sincronização um sinal de realimentação externo nunca tomaria conhecimento desse valor inicial. Essa validação, incluindo o caso do `DFlipFlop` realimentado através de si mesmo para formar um *flip-flop toggle* (Seção 4.9), é apresentada na Seção 5.1.6. A Figura 13 mostra a estrutura mestre-escravo com as linhas correspondentes do Algoritmo 10.

Figura 13 – *Flip-flop* D mestre-escravo, com as linhas do Algoritmo 10.



Fonte: elaborada pelo autor (2026).

4.8.3 `DFlipFlopRegister`: um Registrador de N *Bits* Real

O `DFlipFlop` apresentado acima opera sobre um único *bit*. Para obter uma contraparte do `BehavioralRegister` (Seção 4.7) que seja, de fato, montada com portas lógicas em um contador de N *bits*, o pacote `components` inclui o `DFlipFlopRegister`, apresentado no Algoritmo 11: N instâncias de `DFlipFlop`, uma por posição de *bit* de `data_in`, todas compartilhando o mesmo sinal de `clock` e recombinadas em um único sinal de N *bits*. A técnica de composição é a mesma já usada pelo `RippleCarryAdder` (Seção 4.5): `BitSlice` projeta cada *bit* individualmente (Linha 9) e `Bus` os recombina (Linha 10).

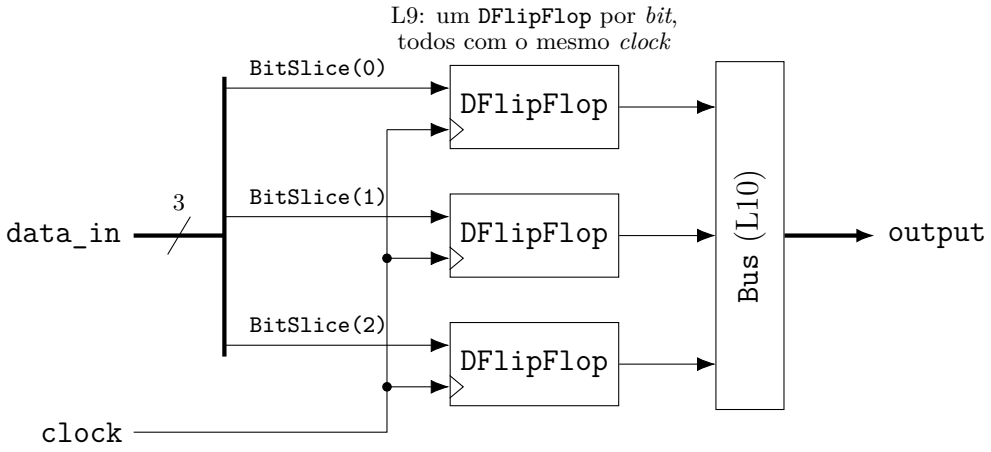
Algoritmo 11 – DFlipFlopRegister (components/d_flip_flop_register.py).

```
1  from signals.bit_slice import BitSlice
2  from signals.bus import Bus
3  from components.d_flip_flop import DFlipFlop
4
5
6  class DFlipFlopRegister:
7      def __init__(self, clock, data_in, feedback_target=None):
8          width = len(data_in.output)
9          self._bits = [DFlipFlop(BitSlice(data_in, i), clock) for i in range(width)]
10         self._output_bus = Bus(self._bits)
11
12         if feedback_target is not None:
13             self._output_bus.add_listener(feedback_target.trigger_change)
14             feedback_target.trigger_change(feedback_target.output,
               ↪  self._output_bus.output)
15
16     @property
17     def output(self):
18         return self._output_bus.output
19
20     def add_listener(self, listener):
21         self._output_bus.add_listener(listener)
```

Fonte: elaborado pelo autor (2026).

A Figura 14 mostra essa composição para N = 3: cada *flip-flop* recebe um *bit* de `data_in` por meio de um `BitSlice` e o mesmo sinal de *clock* (Linha 9 do Algoritmo 11), e o `Bus` criado na Linha 10 reúne as três saídas.

Figura 14 – `DFlipFlopRegister` de 3 *bits*, com as linhas do Algoritmo 11.



Fonte: elaborada pelo autor (2026).

O construtor do `DFlipFlopRegister` tem exatamente a mesma assinatura do `BehavioralRegister` (`clock`, `data_in`, `feedback_target`), o que permite trocar um pelo outro em qualquer circuito de exemplo sem alterar mais nada além do nome da classe instanciada, como no contador da Seção 4.9. Isso torna direta a comparação entre a abstração e sua contraparte real, validada na Seção 5.1.6.

4.9 Composição de Circuitos

Os componentes descritos nas seções anteriores foram combinados para construir e validar os circuitos de exemplo a seguir. Diferentemente dos algoritmos apresentados até aqui, que reproduzem trechos da biblioteca em si, os exemplos a seguir mostram código de uso: código escrito pelo estudante para compor um circuito a partir dos componentes já prontos.

4.9.1 Meio-Somador

Um meio-somador é obtido diretamente pela composição de uma porta `Xor` (soma) e uma porta `And` (*carry*) sobre as mesmas duas entradas de um *bit*, como mostra o Exemplo 1.

Exemplo 1 – Meio-somador composto a partir de Xor e And.

```
1 bx, by = BitArray([x]), BitArray([y])
2 soma = Xor(bx, by)
3 carry = And(bx, by)
```

Fonte: elaborado pelo autor (2026).

4.9.2 Somador Completo

O mesmo meio-somador, estendido com um terceiro sinal de *carry* de entrada, é o próprio `FullAdder` apresentado no Algoritmo 3: basta instanciá-lo com três sinais de 1 *bit* (`a`, `b`, `carry_in`) e ler `.sum` e `.carry_out`.

4.9.3 Contador Binário Realimentado com `BehavioralAdder`

Esta é a versão mais simples do contador, usada quando o foco é demonstrar o registrador e a realimentação. No Exemplo 2, um `BehavioralAdder` soma continuamente uma constante (`001`) ao valor atual do próprio contador, e um `BehavioralRegister` captura esse novo valor a cada pulso de *clock*, devolvendo-o ao mesmo sinal que alimenta o somador.

Exemplo 2 – Contador binário de N *bits* com BehavioralAdder.

```
increment = BitArray([False, False, True])        # constante 001
counter_value = BitArray([False, False, False])   # valor atual do contador

adder = BehavioralAdder(increment, counter_value)      # soma = counter_value + 001
clock = BitArray(False)
counter = BehavioralRegister(clock, adder, feedback_target=counter_value)  #
↪  realimenta
```

Fonte: elaborado pelo autor (2026).

A Figura 3 ilustra a cadeia de propagação desse circuito: a cada pulso de *clock*, o `BehavioralRegister` lê a saída atual do `BehavioralAdder`, atualiza `counter_value`, e essa atualização dispara automaticamente um novo cálculo do `BehavioralAdder`, deixando o circuito pronto para o próximo pulso.

4.9.4 Contador Binário Realimentado com Portas Lógicas

Substituindo o `BehavioralAdder` pelo `RippleCarryAdder`, como no Exemplo 3, o mesmo contador passa a ter a soma montada inteiramente com portas lógicas, e o *overflow* do 8º pulso deixa de depender de `pad_binary_array`, aparecendo diretamente no `carry_out` do somador.

Exemplo 3 – Contador binário de N *bits* com somador de portas lógicas.

```
increment = BitArray([False, False, True])
counter_value = BitArray([False, False, False])
adder = RippleCarryAdder(counter_value, increment)

clock = BitArray(False)
counter = BehavioralRegister(clock, adder.sum, feedback_target=counter_value)
```

Fonte: elaborado pelo autor (2026).

4.9.5 Contador Binário Realimentado com Registrador Real

Trocando também o `BehavioralRegister` pelo `DFlipFlopRegister` (Seção 4.8.3), como no Exemplo 4, o mesmo contador passa a ser montado do início ao fim só com portas lógicas, sem nenhuma abstração de borda de *clock* em Python: tanto a soma quanto o armazenamento do valor entre pulsos são reproduzidos a partir de *latches* e portas. O sinal de `clock` precisa ser um `BitArray` de 1 *bit* em lista (`[False]`, e não `False`), pois agora ele alimenta diretamente portas lógicas (o `Not` de cada `DFlipFlop`), que operam sobre listas de *bits*; o `BehavioralRegister`, por sua vez, aceita as duas formas.

Exemplo 4 – Contador binário de N *bits* com registrador real (DFlipFlopRegister).

```
increment = BitArray([False, False, True])
counter_value = BitArray([False, False, False])
adder = RippleCarryAdder(counter_value, increment)

clock = BitArray([False])
counter = DFlipFlopRegister(clock, adder.sum, feedback_target=counter_value)
```

Fonte: elaborado pelo autor (2026).

Esse contador é validado e comparado ao seu equivalente com `BehavioralRegister` na Seção 5.1.6.

4.9.6 *Flip-Flop Toggle*

O mesmo padrão de realimentação, substituindo o somador por uma porta `Xor` com uma constante `1`, produz um *flip-flop* do tipo *toggle*, cuja saída se inverte a cada pulso de *clock*, como mostra o Exemplo 5.

Exemplo 5 – *Flip-flop toggle*.

```
one = BitArray([True])
clock = BitArray(False)
signal = BitArray([False])

toggle_logic = Xor(signal, one)
flip_flop = BehavioralRegister(clock, toggle_logic, feedback_target=signal)
```

Fonte: elaborado pelo autor (2026).

Esses circuitos, junto com o comportamento dos três contadores em oito ou nove pulsos de *clock* (incluindo o *overflow*), são validados e discutidos no Capítulo 5.

4.10 Tecnologias Utilizadas

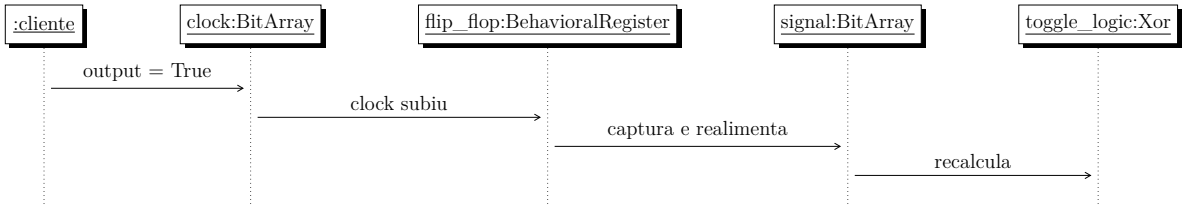
O simulador foi implementado em Python 3, utilizando exclusivamente recursos da biblioteca padrão da linguagem, sem nenhuma dependência externa. Essa escolha reforça o objetivo do trabalho: o estudante pode executar e modificar o simulador com qualquer instalação padrão de Python, sem a necessidade de instalar ou configurar ferramentas adicionais. Cada módulo e cada classe da biblioteca possui uma *docstring* descrevendo seu papel, de modo que a documentação de uso básico esteja disponível diretamente no código (por exemplo, via `help(BitArray)`), complementando a descrição arquitetural apresentada neste capítulo. Os testes automatizados descritos na Seção 5.2 também usam apenas o módulo `unittest` da biblioteca padrão.

4.11 A *Engine* de Propagação

Reunindo o que as seções anteriores apresentaram separadamente, a *engine* deste simulador não é um módulo à parte: é o próprio contrato descrito na Seção 4.2 (`output`, `add_listener`, `trigger_change`), repetido em cada classe. Não existe um laço central de simulação, um agendador, nem uma noção de tempo ou de "ciclo de *clock* global": a propagação inteira acontece dentro da pilha de chamadas do próprio Python, disparada pela primeira atribuição feita pelo código cliente (tipicamente, alternar um `BitArray` de *clock*).

A Figura 15 traduz essa ideia para o caso concreto do *flip-flop toggle* (Seção 4.9), reproduzindo, como diagrama de sequência UML, o mesmo log real de propagação apresentado na Seção 5.1.4: uma única atribuição externa (`clock.output = True`) desencadeia uma cadeia de três chamadas de `trigger_change` em sequência, cada uma completada antes que a próxima comece, até o circuito se estabilizar.

Figura 15 – Diagrama de sequência de um pulso de *clock* no *flip-flop toggle*.



Fonte: elaborada pelo autor (2026), com base no log de execução real da Seção 5.1.4.

Três decisões de projeto, tomadas em conjunto, definem o comportamento observável dessa *engine*:

- **Notificação em cadeia síncrona**: quando `output` de um componente muda, ele chama, um a um, os *listeners* registrados (Linha 16 do Algoritmo 1, `for listener in self._listeners`), e cada chamada de *listener* pode, por sua vez, disparar uma nova cadeia de notificações antes de retornar. Isso significa que a propagação de uma única mudança de sinal é resolvida inteiramente antes que o código cliente recupere o controle: não há filas de eventos nem execução assíncrona;

- **Ordem de notificação**: os *listeners* de um mesmo componente são percorridos na ordem em que foram registrados. Para os circuitos combinacionais e para os circuitos sequenciais construídos sobre o `BehavioralRegister`, a ordem de notificação não altera o resultado final, pois cada componente recalcula seu próprio estado de forma independente a partir das entradas atuais, e não da ordem de chegada das notificações. Ela deixa de ser um detalhe inofensivo, porém, quando dois componentes que deveriam se "revezar" (como o *latch* mestre e o *latch* escravo do `DFlipFlop`, Seção 4.8) dependem do mesmo sinal de `clock`: nesse caso, a ordem determina qual dos dois reage primeiro a uma mudança, e é por isso que `not_clock` é registrado como *listener* de `clock` antes das portas do *latch* escravo (que só existem depois que o *latch* mestre, e portanto `not_clock`, já foi construído): como a notificação segue a ordem de registro, o mestre sempre fecha antes de o escravo abrir. Essa validação é apresentada na Seção 5.1.6;

- **Filtragem por igualdade**: cada componente só recalcula seu `_output` e notifica seus *listeners* quando o novo valor calculado difere do anterior (Linhas 13 e 14 do Algoritmo 1). Sem esse filtro, um laço de realimentação combinacional genuíno (como o do `SRLatch`, Seção 4.8) reentraria indefinidamente na mesma cadeia de chamadas.

Essas três regras bastam para explicar tanto os circuitos que funcionam quanto os que podem não estabilizar: um circuito acíclico (qualquer composição de portas sem realimentação) sempre termina de propagar em um número finito de passos, pois cada

componente é notificado uma vez por mudança de entrada; um circuito com realimentação que passa por um `BehavioralRegister` também termina, porque o `BehavioralRegister` só propaga adiante no instante controlado da borda de *clock*, e não a cada mudança intermediária de `data_in`; um circuito com realimentação através de *latches* (o `SRLatch` ou o `DFlipFlop` mestre-escravo, Seção 4.8) também termina, mas depende, além do filtro por igualdade, de que a ordem de notificação respeite a ordem em que os componentes foram construídos (mestre antes de escravo), para que a alternância de transparência entre os dois *latches* funcione como um mecanismo de corte equivalente ao do `BehavioralRegister`. Se essa ordem não fosse respeitada, o circuito degeneraria em um laço puramente combinacional com uma única inversão, sem ponto fixo algum, independentemente do filtro por igualdade.

5 Resultados e Discussão

Este capítulo apresenta os resultados obtidos com a execução dos circuitos de exemplo descritos no Capítulo 4, comparando a saída produzida pelo simulador com o comportamento teoricamente esperado. Em seguida, descrevem-se os testes automatizados que verificam esses resultados e os recursos de observação e depuração que o simulador oferece, comparados aos de outras ferramentas, e discutem-se as limitações da abordagem.

5.1 Validação dos Circuitos Implementados

Esta seção percorre, em ordem crescente de complexidade, a validação de cada circuito descrito no Capítulo 4: da propagação reativa em uma única porta lógica até os três contadores binários e o *flip-flop toggle*, incluindo a validação do registrador construído com *latches*.

5.1.1 Porta NOT Reativa

O primeiro teste verifica se uma porta lógica recalcula sua saída automaticamente quando sua entrada muda, sem que nenhum código externo solicite esse recálculo. Partindo de `a = BitArray([False, True, False])` e `not_a = Not(a)`, obteve-se o resultado apresentado na Tabela 3.

Tabela 3 – Propagação reativa através de uma porta NOT.

Evento	**not_a.output**
Estado inicial (`a = [F, T, F]`)	`[T, F, T]`
Após `a.output = [T, T, T]`	`[F, F, F]`

Fonte: elaborada pelo autor (2026), com base na execução do simulador.

O resultado confirma que a alteração de `a.output` propagou-se automaticamente para `not_a`, sem qualquer chamada explícita a um método de recálculo, validando o mecanismo de notificação descrito na Seção 4.3.

5.1.2 Meio-Somador e Somador Completo

O meio-somador (`Xor` para soma, `And` para *carry*) foi testado com as quatro combinações possíveis de entrada de 1 *bit*, reproduzindo exatamente a tabela-verdade esperada. O mesmo teste foi repetido para o `FullAdder` (Seção 4.5), incluindo agora um *carry* de entrada. Os dois resultados são apresentados na Tabela 4.

Tabela 4 – Saída do meio-somador e do somador completo comparadas à tabela-verdade esperada.

A	B	C_{in}	Soma	*Carry-out*
Meio-somador (sem carry de entrada)				
0	0	–	0	0
0	1	–	1	0
1	0	–	1	0
1	1	–	0	1
Somador completo (8 combinações de A, B e C_{in})				
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Fonte: elaborada pelo autor (2026), com base na execução do simulador.

Todos os valores obtidos corresponderam exatamente ao esperado pela álgebra booleana (Seção 2.1.1), tanto para o meio-somador (linhas superiores, sem *carry* de entrada) quanto para o somador completo.

5.1.3 Registrador Simples

Um `BehavioralRegister` sem realimentação foi usado para validar a detecção de borda de subida isoladamente: antes de qualquer pulso de *clock*, `register.output` permanece `None`; após `clock.output = True`, o valor de `data_in.output` no instante da borda é capturado e mantido, mesmo que `data_in.output` mude posteriormente sem um novo pulso de *clock*, e mesmo quando o *clock* volta a falso (borda de descida). O mesmo comportamento foi verificado com o *clock* representado tanto como booleano (`BitArray(False)`) quanto como lista de um *bit* (`BitArray([False])`). Esse comportamento confirma que o `BehavioralRegister` reage apenas à borda de subida do sinal de *clock*, e não a mudanças na entrada de dados.

5.1.4 Contador Binário e *Flip-Flop Toggle*

Os dois circuitos com laço de realimentação foram validados pulsando o *clock* repetidamente e comparando a sequência de saídas com a esperada para, respectivamente, um contador binário crescente (usando `BehavioralAdder`) e um *flip-flop* que inverte de estado a cada pulso. As duas sequências obtidas são apresentadas na Tabela 5.

Tabela 5 – Saída do contador de 3 *bits* e do *flip-flop toggle* ao longo de sucessivos pulsos de *clock*.

Pulso	**Contador (3 bits)**	***Flip-flop toggle***
1	001	1
2	010	0
3	011	1
4	100	0
5	101	1
6	110	0
7	111	—

Fonte: elaborada pelo autor (2026), com base na execução do simulador.

Ambas as sequências corresponderam exatamente ao esperado: o contador incrementa uma unidade a cada pulso, e o *flip-flop toggle* alterna seu estado, confirmando que o `BehavioralRegister` realimentado é capaz de sustentar um circuito com estado persistente ao longo de múltiplos ciclos de *clock*.

Para tornar visível a ordem real de propagação dentro de um único pulso, um *listener* de registro (*log*) foi anexado a cada componente do circuito do *flip-flop toggle*. O Quadro 1 mostra a saída obtida para um único pulso de *clock*, evidenciando como o sinal se propaga em cadeia até se estabilizar.

Quadro 1 – Log de propagação de um pulso de *clock* no *flip-flop toggle*.

```
estado antes do pulso: signal = [False]
pulso de clock (False -> True):
  toggle_logic: [True] -> [False]
  signal:       [False] -> [True]
  flip_flop:    [False] -> [True]
  clock:        False -> True
estado apos o pulso: signal = [True]
```

Fonte: elaborado pelo autor (2026), com base na execução do simulador.

A ordem desse log só faz sentido à luz de como cada linha é impressa: o *listener* de log foi registrado em cada sinal **depois** do *listener* funcional que já existia desde a Seção 4.9 (o próprio `BehavioralRegister`, no caso de `clock` e `flip_flop`; o próprio `Xor`, no caso de `signal`), e a *engine* notifica *listeners* na ordem em que foram registrados (Seção 4.11). Assim, a atribuição `clock.output = True` primeiro dispara o `BehavioralRegister` (`flip_flop`), que lê o valor já estabilizado de `toggle_logic` (`True`) e o propaga para `signal`, que por sua vez dispara o recálculo de `toggle_logic` (`False`) antes de qualquer linha ser impressa; só então, à medida que cada chamada de `trigger_change` retorna, os quatro *listeners* de log dessa cadeia disparam em ordem inversa à da causa: primeiro o de `toggle_logic` (o mais interno, que recalculou por

último), depois `signal`, depois `flip_flop`, e por fim o de `clock` (o mais externo, que só é notificado quando a pilha de chamadas termina de desenrolar). Nenhum desses quatro componentes é notificado mais de uma vez, o que confirma que o circuito se estabiliza em uma única passada de propagação para esse caso.

5.1.5 Contador Binário Montado Só com Portas Lógicas

O mesmo contador foi refeito trocando o `BehavioralAdder` pelo `RippleCarryAdder` (Seção 4.5), permitindo observar também o sinal `carry_out` a cada pulso, como mostra a Tabela 6.

Tabela 6 – Contador de 3 *bits* montado com RippleCarryAdder, com carry_out.

Pulso	**Contador (3 bits)**	**carry_out**
1	001	0
2	010	0
3	011	0
4	100	0
5	101	0
6	110	0
7	111	1
8	000	0
9	001	0

Fonte: elaborada pelo autor (2026), com base na execução do simulador.

O resultado reproduziu exatamente a sequência do contador com `BehavioralAdder`, com uma diferença importante: no pulso 7, assim que o valor `111` é realimentado para a entrada do somador, `carry_out` sobe para `1`, sinalizando que a próxima soma (`111 + 001`) excede 3 *bits*. No pulso 8, o `BehavioralRegister` captura a soma já "estourada" (`000`), e `carry_out` volta a `0` porque a nova entrada do somador (`000`) não gera mais *overflow*. Esse resultado confirma, na prática, o que foi projetado na Seção 4.5: o *overflow* de um somador montado com portas lógicas aparece naturalmente em um sinal de *carry* separado, sem exigir nenhum tratamento especial em código.

5.1.6 Registrador Real com *Latches*

O *latch* SR (Seção 4.8) foi validado com a sequência clássica de *set*, *hold* e *reset*, reproduzindo exatamente o comportamento esperado, como mostra a Tabela 7.

Tabela 7 – Validação do *latch* SR (NOR cruzado).

S	R	Q
1	0	`[True]` (*set*)
0	0	`[True]` (*hold*, mantém o valor anterior)
0	1	`[False]` (*reset*)
0	0	`[False]` (*hold*, mantém o valor anterior)

Fonte: elaborada pelo autor (2026), com base na execução do simulador.

O `DFlipFlop` (Seção 4.8) foi então validado com um sinal D independente, alternando o *clock* e o próprio D em momentos distintos para confirmar a sensibilidade exclusiva à borda de subida. A saída produzida pelo programa `experiments.py` é apresentada no Quadro 2.

Quadro 2 – Validação do `DFlipFlop` isolado.

```
D=1, clock sobe -> Q=[True] (capturou D)
D muda p/ 0 com clock ainda em 1 -> Q=[True] (nao muda)
clock desce (1->0) -> Q=[True] (flip-flop so muda na borda de subida)
clock sobe de novo com D=0 -> Q=[False] (capturou o novo D)
```

Fonte: elaborado pelo autor (2026), com base na execução do simulador.

O resultado confirma que o `DFlipFlop` reproduz, a partir de portas lógicas reais, o mesmo comportamento observável do `BehavioralRegister`: a saída só se atualiza no instante da borda de subida do *clock*, permanecendo estável mesmo que D mude enquanto o *clock* está em nível alto ou baixo.

O *flip-flop toggle* da Seção 4.9 foi então reproduzido substituindo o `BehavioralRegister` pelo `DFlipFlop` (isto é, realimentando a própria saída, invertida, de volta para a entrada D). Diferentemente do `BehavioralRegister`, que quebra qualquer ciclo de realimentação lendo `data_in` apenas no instante exato da borda de *clock*, o `DFlipFlop` depende inteiramente da separação de transparência entre o *latch* mestre e o *latch* escravo (Seção 4.8) para que essa realimentação não degenere em um laço puramente combinacional: a ordem de notificação do simulador (Seção 4.11) garante que `not_clock` feche o *latch* mestre antes de as portas do *latch* escravo serem notificadas, preservando essa separação a cada pulso de *clock*. O resultado é apresentado na Tabela 8.

Tabela 8 – Saída do *flip-flop toggle* montado com DFlipFlop.

Pulso	**Saída**
Estado inicial	1
1	0
2	1
3	0
4	1
5	0
6	1

Fonte: elaborada pelo autor (2026), com base na execução do simulador.

A sequência alterna estritamente a cada pulso, com o mesmo padrão observado para o *flip-flop toggle* construído sobre o `BehavioralRegister` (Seção 5.1.4), a menos de uma defasagem de fase: o estado inicial começa em 1 em vez de 0. Em um circuito físico, o valor de Q antes do primeiro pulso de *clock* seria indeterminado; no simulador, ele é sempre o mesmo, mas arbitrário: decorre do valor inicial atribuído ao `BitArray` de proxy usado para fechar o laço do `SRLatch` (`[False]`, Linha 7 do Algoritmo 8), que faz com que $Q = \overline{R}$ na construção e, portanto, $Q = 1$ enquanto o *latch* não está habilitado. Trata-se de um artefato da inicialização, e não de um defeito na propagação.

Para validar esse comportamento em uma escala maior do que um único *flip-flop*, o contador de 3 *bits* montado com `RippleCarryAdder` (Seção 5.1.5) foi reconstruído trocando também o `BehavioralRegister` pelo `DFlipFlopRegister` (Seção 4.8.3), um registrador de N *bits* montado inteiramente com portas lógicas. A Tabela 9 compara o resultado, lado a lado, com o contador de referência (que usa o `BehavioralRegister`).

Tabela 9 – Contador de 3 *bits* com BehavioralRegister comparado ao mesmo contador com DFlipFlopRegister.

Pulso	**BehavioralRegister**	**DFlipFlopRegister**
Estado inicial	`None`	111
1	001	000
2	010	001
3	011	010
4	100	011
5	101	100
6	110	101
7	111 (carry_out=1)	110
8	000	111 (carry_out=1)
9	001	000

Fonte: elaborada pelo autor (2026), com base na execução do simulador.

As duas sequências são idênticas, inclusive no pulso em que o *overflow* ocorre, a menos da mesma defasagem de uma posição já explicada para o *flip-flop toggle*: o

`DFlipFlopRegister` começa em `111` porque cada um dos três `DFlipFlop` que o compõem parte do mesmo valor inicial arbitrário ($Q = 1$) antes do primeiro pulso de *clock*. Esse resultado estende a validação de um único *bit* realimentado para um registrador completo de N *bits*, sem que nenhuma das duas classes (`BehavioralRegister` e `DFlipFlopRegister`) precisasse de qualquer ajuste específico para lidar com múltiplos *bits*: o `DFlipFlopRegister` reaproveita, sem alterações, a mesma técnica de composição por `BitSlice` e `Bus` já usada pelo `RippleCarryAdder` (Seção 4.5). Vale notar o escopo dessa validação: ela comprova que este caso específico, exaustivamente testado, estabiliza corretamente, mas não que toda topologia de realimentação combinacional através de *latches* estabiliza sob a mesma convenção de ordenação, ressalva retomada na Seção 5.4.

5.2 Testes Automatizados

Os resultados apresentados na seção anterior foram inicialmente obtidos executando o programa `experiments.py` e comparando visualmente sua saída com o comportamento esperado. Para que essa validação seja reproduzível e não dependa de inspeção manual, cada um desses resultados foi transformado em um caso de teste automatizado no arquivo `tests/test_circuitos.py`, escrito com o módulo `unittest` da biblioteca padrão de Python. Os testes verificam, por meio de asserções, a porta NOT reativa, o meio-somador, o somador completo nas oito combinações de entrada, o `RippleCarryAdder` de 3 *bits* em todas as 64 combinações de operandos (incluindo o `carry_out`), a captura do `BehavioralRegister` apenas na borda de subida, os três contadores ao longo de nove pulsos de *clock*, o *latch* SR, o `DFlipFlop` isolado e os dois *flip-flops toggle*.

Um dos testes cobre especificamente o defeito de detecção de borda do `BehavioralRegister` descrito na Seção 4.7: com o *clock* representado como lista (`BitArray([False])`), ele verifica que o valor capturado não muda na borda de descida. Esse teste falhava com a versão anterior do componente e passa com a versão atual. Os treze testes são executados com o comando `python -m unittest discover -s tests -t .` e, na versão final do simulador, todos passam.

5.3 Recursos de Observação e Depuração

Como cada componente do simulador é um objeto Python comum, e a propagação acontece por chamadas de função entre esses objetos (Seção 4.11), o estudante pode observar e interrogar o circuito usando a mesma linguagem com que o construiu, sem nenhuma ferramenta adicional. Dois desses recursos já foram usados neste capítulo: a impressão do estado a cada ciclo de *clock* (Tabelas 5 a 9) e o registro da ordem de propagação dentro de um único pulso (Quadro 1). Esta seção apresenta outros cinco, todos reunidos no programa `recursos_observacao.py` e executados para produzir as saídas

reproduzidas a seguir, e compara esses recursos aos oferecidos pelas ferramentas discutidas no Capítulo 3.

5.3.1 Estados Transitórios (*Glitches*)

Em um somador estrutural, o resultado não aparece de uma só vez: cada estágio só pode calcular sua soma depois de receber o *carry* do estágio anterior, e a saída passa por valores intermediários até se estabilizar. No simulador, basta registrar um *listener* na saída do somador para observar esses valores. O Exemplo 6 monta, sobre as mesmas entradas, um `RippleCarryAdder` e um `BehavioralAdder` de 4 *bits*, e altera a entrada `b` de forma que o *carry* precise atravessar três estágios ($0111 + 0001$).

Exemplo 6 – Observação dos estados transitórios de um somador.

```
a = BitArray([False, True, True, True])      # 0111
b = BitArray([False, False, False, False])   # 0000
estrutural = RippleCarryAdder(a, b).sum
comportamental = BehavioralAdder(a, b)
estrutural.add_listener(lambda old, new: print("estrutural:    ", bits(new)))
comportamental.add_listener(lambda old, new: print("comportamental:", bits(new)))
b.output = [False, False, False, True]       # 0111 + 0001 = 1000
```

Fonte: elaborado pelo autor (2026).

O Quadro 3 mostra que a saída do somador estrutural passou por três valores intermediários antes de chegar a `1000`, enquanto o somador comportamental, que calcula a soma de uma vez em Python, passou diretamente ao resultado final.

Quadro 3 – Saídas observadas durante a propagação do Exemplo 6.

```
estrutural:     0110
estrutural:     1110
estrutural:     1010
estrutural:     1000
comportamental: 1000
```

Fonte: elaborado pelo autor (2026), com base na execução do simulador.

Esse resultado torna visível, de forma concreta, a diferença entre as duas descrições e a natureza sequencial da propagação do *carry* em um *ripple-carry adder*. É preciso, porém, interpretá-lo com cuidado: como o simulador não modela atraso de propagação (Seção 5.4), a sequência exata de valores intermediários é determinada pela ordem de notificação dos *listeners*, e não por atrasos físicos. Ela ilustra qualitativamente o fenômeno, mas não prevê os transitórios de um circuito real específico.

5.3.2 Atividade de Chaveamento

A mesma técnica permite contar quantas vezes os sinais internos de um circuito mudam durante uma operação, medida conhecida como atividade de chaveamento, à qual se relacionam o tempo de estabilização e o consumo dinâmico de energia de circuitos reais. O Exemplo 7 usa a lista `full_adders` do `RippleCarryAdder` (Algoritmo 4) para registrar um contador em cada saída de soma e de *carry* dos oito estágios de um somador de 8 *bits*, e soma 1 a dois valores iniciais diferentes.

Exemplo 7 – Contagem da atividade de chaveamento de um somador de 8 *bits*.

```
for inicial in ([False] * 8, [False] + [True] * 7):
    a = BitArray(inicial)
    b = BitArray([False] * 8)
    somador = RippleCarryAdder(a, b)
    mudancas = []
    for estagio in somador.full_adders:
        estagio.sum.add_listener(lambda old, new: mudancas.append(1))
        estagio.carry_out.add_listener(lambda old, new: mudancas.append(1))
    b.output = [False] * 7 + [True]          # soma 1
    print(f"{bits(inicial)} + 00000001: {len(mudancas)} mudança(s)")
```

Fonte: elaborado pelo autor (2026).

Como mostra o Quadro 4, somar 1 a `00000000` altera um único sinal interno, enquanto somar 1 a `01111111`, que obriga o *carry* a percorrer sete estágios, altera quinze. O estudante pode, assim, verificar experimentalmente por que o pior caso de um *ripple-carry adder* é a propagação do *carry* pela cadeia inteira.

Quadro 4 – Atividade de chaveamento medida pelo Exemplo 7.

```
00000000 + 00000001: 1 mudança(s)
01111111 + 00000001: 15 mudança(s)
```

Fonte: elaborado pelo autor (2026), com base na execução do simulador.

5.3.3 Sondas com Asserções

Um *listener* não precisa apenas imprimir valores: ele pode verificar uma condição e interromper a simulação quando ela é violada, funcionando como uma sonda que vigia o circuito continuamente. O Exemplo 8 registra, nas entradas de um `SRLatch`, uma sonda que detecta a combinação inválida $S = R = 1$ (Seção 4.8) no exato instante em que ela ocorre.

Exemplo 8 – Sonda que detecta a combinação inválida do *latch* SR.

```
s, r = BitArray([False]), BitArray([False])
latch = SRLatch(s, r)

def sonda(old, new):
    assert not (s.output[0] and r.output[0]), "combinação inválida S = R = 1"

s.add_listener(sonda)
r.add_listener(sonda)
s.output = [True]
r.output = [True]    # dispara a sonda
```

Fonte: elaborado pelo autor (2026).

A primeira atribuição é aceita normalmente; a segunda interrompe a execução com a mensagem definida pelo estudante, como mostra o Quadro 5. Por ser uma exceção comum de Python, ela pode ser capturada, registrada ou usada para encerrar um teste automatizado.

Quadro 5 – Saída do Exemplo 8.

```
S=1 R=0 -> Q = 1
AssertionError: combinação inválida S = R = 1
```

Fonte: elaborado pelo autor (2026), com base na execução do simulador.

5.3.4 Caminho da Propagação

Como a propagação de uma mudança é uma cadeia de chamadas de função (Seção 4.11), a pilha de chamadas do Python registra, a cada instante, exatamente por quais componentes o sinal passou. O Exemplo 9 aproveita esse fato: um *listener* no `carry_out` de um somador de 2 *bits* percorre a pilha com o módulo `inspect` da biblioteca padrão e imprime o tipo de cada componente encontrado.

Exemplo 9 – Reconstrução do caminho percorrido por uma mudança de sinal.

```
a = BitArray([True, True])                    # 11
b = BitArray([False, False])                  # 00
somador = RippleCarryAdder(a, b)

def caminho(old, new):
    componentes = [type(f.frame.f_locals["self"]).__name__
                   for f in inspect.stack()
                   if "self" in f.frame.f_locals]
    print(" -> ".join(reversed(componentes)))

somador.carry_out.add_listener(caminho)
b.output = [False, True]                      # 11 + 01
```

Fonte: elaborado pelo autor (2026).

O resultado, apresentado no Quadro 6, é o caminho do *carry* pelo circuito: a mudança em `b` chega ao *bit* menos significativo por um `BitSlice`, gera um *carry* na porta `And` (`and_ab`) e na porta `Or` (`carry_out`) do primeiro estágio e é repassada às portas `And` (`and_xor_carry`) e `Or` do segundo estágio (Figura 7). O mesmo efeito pode ser obtido de forma interativa colocando uma chamada a `breakpoint()` dentro do *listener*: a simulação é pausada no meio da propagação e o depurador padrão do Python permite inspecionar cada componente da pilha.

Quadro 6 – Caminho da propagação impresso pelo Exemplo 9.

```
BitArray -> BitSlice -> And -> Or -> And -> Or
```

Fonte: elaborado pelo autor (2026), com base na execução do simulador.

5.3.5 Equivalência entre Descrições

Por fim, como o circuito é um objeto Python, ele pode ser instanciado dentro de laços, o que permite gerar tabelas-verdade completas ou comparar automaticamente duas descrições do mesmo componente. O Exemplo 10 verifica, para todas as 64 combinações de dois operandos de 3 *bits*, que o somador estrutural e o comportamental produzem a mesma soma; ao final da execução, o programa imprime “64 combinações verificadas: as duas descrições são equivalentes”. Esse tipo de verificação aproxima o estudante, ainda em uma linguagem conhecida, da prática de verificação usada no projeto de *hardware*, em que uma descrição estrutural é comparada a um modelo de referência comportamental.

Exemplo 10 – Verificação exaustiva de equivalência entre as descrições estrutural e comportamental.

```
for x, y in product(range(8), repeat=2):
    a, b = BitArray(to_bits(x, 3)), BitArray(to_bits(y, 3))
    assert RippleCarryAdder(a, b).sum.output == BehavioralAdder(a, b).output
print("64 combinações verificadas: as duas descrições são equivalentes")
```

Fonte: elaborado pelo autor (2026).

5.3.6 Comparação com Outras Ferramentas

Vários desses recursos têm equivalentes em outras ferramentas: simuladores de HDL oferecem diagramas de tempo e asserções; o CircuitVerse combina atrasos configuráveis por componente com um diagrama de tempo, que permite observar transitórios e condições de corrida; e o Logisim permite avançar a propagação passo a passo pela interface. A diferença está na forma como são oferecidos. Nas ferramentas gráficas e nos simuladores de HDL, cada recurso é uma funcionalidade pronta, com comportamento definido pela ferramenta; no simulador deste trabalho, todos eles são escritos pelo próprio estudante, em poucas linhas de Python, sobre o mesmo mecanismo de *listeners* que faz o circuito funcionar, e podem ser combinados ou adaptados livremente. A Tabela 10 resume essa comparação para as ferramentas discutidas no Capítulo 3.

Tabela 10 – Recursos de observação e depuração em diferentes ferramentas.

Recurso	**Logisim**	**Circuit-Verse**	**DigitalJS**	**Sim. HDL**	**MyHDL/ PyRTL**	**Este trabalho**
Valores a cada ciclo de *clock*	I	F	F	F	F	Py
Transitórios dentro de um passo	P	F	–	P	–	Py
Sondas e asserções	–	F	–	F	Py	Py
Atividade de chaveamento	–	–	–	P	Py	Py
Caminho da propagação	–	–	–	P	–	Py
Equivalência entre descrições	–	F	–	F	Py	Py

I: pela interface gráfica; F: recurso pronto da ferramenta; P: parcial ou apenas em algumas versões/ferramentas; Py: escrito em Python pelo próprio usuário; –: não oferecido.
Fonte: elaborada pelo autor (2026), com base nas ferramentas descritas no Capítulo 3.

A coluna "Sim. HDL" agrupa simuladores como GHDL e ModelSim, cujos recursos variam entre si; as indicações "P" refletem recursos como a visualização de ciclos delta e a cobertura de alternância de sinais (*toggle coverage*), disponíveis apenas em parte desses simuladores. Para MyHDL e PyRTL, "Py" indica que o recurso pode ser escrito em Python sobre os sinais da biblioteca, mas a propagação em si continua encapsulada em

seu núcleo de simulação (Seção 3.2), o que impede, por exemplo, reconstruir o caminho de uma mudança porta a porta. O traço comum da última coluna é, portanto, o argumento central deste trabalho visto sob outro ângulo: o mecanismo de simulação é parte do código que o estudante lê e escreve.

5.4 Limitações da Abordagem

A validação dos circuitos de exemplo também expôs limitações da implementação atual, relevantes tanto para o uso didático do simulador quanto para trabalhos futuros:

- As portas `Or`, `And` e `Xor` assumem que as duas entradas possuem o mesmo número de *bits*, comparando-as índice a índice; não há verificação explícita desse pré-requisito, e um descompasso de tamanho resulta em erro de execução (`IndexError`) em vez de uma mensagem de erro didática;
- O `BehavioralRegister` suporta apenas um único sinal de `clock`, sem sinais adicionais como *reset* assíncrono ou *enable* independente do *clock*;
- O simulador não modela atraso de propagação: todas as portas reagem instantaneamente, no mesmo instante lógico. Isso é suficiente para os circuitos combinacionais e para todos os circuitos sequenciais testados neste trabalho, incluindo o `DFlipFlop` realimentado sobre si mesmo (Seção 5.1.6), desde que a ordem de notificação dos *listeners* respeite a ordem de construção dos componentes; a ausência de um modelo de tempo simulado permanece, ainda assim, uma limitação em relação a um simulador de eventos discretos completo, e não há garantia de que topologias de realimentação através de *latches* mais complexas do que as testadas aqui estabilizem sob a mesma convenção de ordenação.

5.5 Discussão

Os resultados obtidos indicam que o modelo de propagação reativa, baseado no padrão *Observer*, é adequado para reproduzir corretamente o comportamento de circuitos digitais combinacionais e sequenciais de pequena escala: em todos os circuitos testados, a saída do simulador correspondeu exatamente ao comportamento esperado, incluindo os dois tratamentos de *overflow* discutidos (o truncamento do `BehavioralAdder`, Seção 4.6, e o `carry_out` do `RippleCarryAdder`, Seção 5.1.5).

A coexistência desses dois somadores no simulador não é uma inconsistência, mas um reflexo do objetivo geral deste trabalho: apoiar o aprendizado dos conceitos introdutórios da disciplina de Circuitos e Técnicas Digitais. O `RippleCarryAdder` representa a forma tecnicamente correta de montar um somador, inteiramente a partir de portas lógicas, e

evidencia como o *overflow* surge da própria topologia do circuito. Já o `BehavioralAdder` demonstra que o simulador permite misturar Python "puro" com o restante do circuito reativo quando o objeto de estudo do momento é outro (por exemplo, o comportamento do registrador), sem obrigar o estudante a montar toda a aritmética com portas lógicas antes de poder experimentar com registradores e realimentação. O mesmo raciocínio vale para o par `BehavioralRegister` e `DFlipFlopRegister`. Os circuitos implementados cobrem, assim, os elementos centrais dessa etapa inicial do currículo: portas lógicas, propagação de sinais, um somador (nas duas variantes) e um componente de memória sensível a *clock* (também nas duas variantes). A implementação inteiramente em Python, sem dependências externas, mantém o ambiente de execução idêntico ao já utilizado pelo estudante na disciplina de Programação Orientada a Objetos, eliminando a barreira de ferramental discutida na motivação (Capítulo 1).

Por outro lado, as limitações apontadas na seção anterior, em particular a ausência de outros componentes combinacionais comuns, como multiplexadores, delimitam o alcance atual do simulador a um conjunto reduzido de conceitos introdutórios, não sendo ainda suficiente para representar circuitos de maior complexidade, como uma Unidade Lógica e Aritmética (ULA) ou um banco de registradores. Essas limitações são retomadas como sugestões de trabalhos futuros no Capítulo 6.

6 Conclusão

Este trabalho partiu de um problema concreto observado na trajetória de estudantes de Ciência da Computação da UFSC: a distância entre a programação em Python, já familiar da disciplina de Programação Orientada a Objetos, e a descrição de *hardware* em VHDL, exigida logo nos primeiros contatos com a disciplina de Circuitos e Técnicas Digitais. Diante disso, o objetivo geral proposto no Capítulo 1 foi desenvolver, em Python, um simulador didático de circuitos digitais baseado em propagação reativa de sinais, capaz de apoiar o aprendizado dos conceitos introdutórios dessa disciplina.

Os cinco objetivos específicos definidos no Capítulo 1 foram atingidos: (i) o barramento de *bits* com notificação automática foi implementado pela classe `BitArray`; (ii) as portas lógicas básicas (`Not`, `And`, `Or`, `Xor`) foram implementadas como componentes reativos; (iii) o componente sequencial `BehavioralRegister`, sensível a borda de subida de *clock* e escrito em lógica Python, foi implementado e validado isoladamente, assim como sua contraparte construída inteiramente com *latches* (`SRLatch`, `DLatch`, `DFlipFlop` e sua extensão a N *bits*, `DFlipFlopRegister`); (iv) a composição desses componentes em circuitos maiores foi demonstrada por meio de um meio-somador, um somador completo, três contadores binários (com o atalho `BehavioralAdder`, com `RippleCarryAdder` e `BehavioralRegister`, e com `RippleCarryAdder` e `DFlipFlopRegister`) e um *flip-flop toggle*; e (v) todos esses circuitos foram validados no Capítulo 5, inclusive por meio de testes automatizados, com a saída do simulador correspondendo exatamente ao comportamento esperado em cada caso, incluindo o tratamento de *overflow* nas suas duas formas (truncamento no `BehavioralAdder` e sinal `carry_out` no `RippleCarryAdder`).

De forma mais ampla, os resultados obtidos sustentam a hipótese levantada na motivação deste trabalho: um simulador implementado inteiramente em Python, sem dependências externas, é capaz de reproduzir corretamente os conceitos introdutórios de circuitos digitais em um ambiente de execução já familiar ao estudante, servindo como uma etapa intermediária de aprendizado antes do contato com HDLs tradicionais como VHDL e Verilog.

6.1 Contribuições

As principais contribuições deste trabalho são:

- um simulador de circuitos digitais funcional, implementado em Python 3 sem dependências externas, organizado em quatro pacotes (`signals`, `gates`, `components`, `utils`) que separam claramente representação de sinais, portas lógicas elementares, componentes compostos e funções auxiliares;

- uma arquitetura reativa baseada no padrão de projeto *Observer*, na qual cada componente atua simultaneamente como observador de suas entradas e sujeito observado por sua saída, propagando mudanças automaticamente ao longo do circuito;
- dois somadores com propósitos didáticos distintos e explicitamente documentados: o `RippleCarryAdder`, montado inteiramente a partir de portas lógicas e cujo *overflow* surge naturalmente da topologia do circuito, e o `BehavioralAdder`, que demonstra a possibilidade de misturar Python "puro" com o restante do circuito reativo quando o objeto de estudo é outra parte do circuito;
- um registrador construído inteiramente com *latches* (`SRLatch`, `DLatch`, `DFlipFlop` mestre-escravo, e sua extensão a N *bits*, `DFlipFlopRegister`), complementando o `BehavioralRegister` e validado, inclusive, na configuração mais exigente testada neste trabalho (um *flip-flop toggle* realimentado através de si mesmo) e em um contador de 3 *bits* completo, comparado lado a lado com o contador equivalente montado sobre o `BehavioralRegister`;
- um conjunto de circuitos de exemplo (meio-somador, somador completo, três contadores binários e um *flip-flop toggle*, este último nas versões com `BehavioralRegister` e com `DFlipFlop` de portas) validados comparando a saída do simulador ao comportamento teoricamente esperado e verificados por um conjunto de testes automatizados;
- um conjunto de recursos de observação e depuração construídos sobre o próprio mecanismo de *listeners* (visualização de estados transitórios, medição de atividade de chaveamento, sondas com asserções, reconstrução do caminho da propagação e verificação de equivalência entre descrições), escritos em poucas linhas de Python e comparados aos recursos equivalentes de outras ferramentas (Seção 5.3).

6.2 Trabalhos Futuros

Como possibilidades de continuidade para este trabalho, destaca-se a inclusão de novos componentes combinacionais, como multiplexadores e decodificadores, além de um sinal de *reset* assíncrono para o `BehavioralRegister`. No plano da *engine* de simulação, seria relevante detectar laços combinacionais inválidos automaticamente e introduzir um modelo de atraso de propagação (ciclos delta, ou um agendador de eventos com tempo simulado), tornando a estabilização de circuitos com realimentação através de *latches* independente de uma convenção específica de ordenação de *listeners* (Seção 5.1.6). Do ponto de vista de usabilidade, a geração de diagramas de tempo (por exemplo, exportando o histórico dos sinais em formato VCD para visualização em ferramentas como o GTKWave), um gerador de *clock* automático (independente da alternância manual de `clock.output`) e

uma interface gráfica/visual para acompanhar a propagação dos sinais em tempo real tornariam o simulador mais acessível a estudantes iniciantes. Por fim, como validação pedagógica mais ampla, propõe-se avaliar o uso do simulador em turmas reais da disciplina de Circuitos e Técnicas Digitais, comparando a curva de aprendizado com a abordagem tradicional baseada diretamente em VHDL.

Referências

ASHENDEN, P. J. *The Designer's Guide to VHDL*. 3. ed. Amsterdam: Morgan Kaufmann, 2008.

AZEVEDO, F. S. T. de. *FPGA como ferramenta de ensino de engenharia*. Monografia (Graduação em Engenharia de Controle e Automação) — Fundação Presidente Antônio Carlos (FUPAC), Conselheiro Lafaiete, MG, 2017. Disponível em: <https://ri.unipac.br/repositorio/wp-content/uploads/2019/09/2017-02-Filipe-Samuel-Teixeira-de-Azevedo.pdf>. Acesso em: 29 out. 2026.

BAINOMUGISHA, E. et al. A survey on reactive programming. *ACM Computing Surveys*, New York, v. 45, n. 4, p. 1–34, ago. 2013. Art. 52. Disponível em: <https://doi.org/10.1145/2501654.2501666>. Acesso em: 29 out. 2026.

BATISTA, E. L. O. *FPGA Emulator Web*. 2021. Repositório de código-fonte. Universidade Federal de Santa Catarina. Disponível em: <https://codigos.ufsc.br/eduardo.batista/fpgaemuweb>. Acesso em: 29 out. 2026.

BINPY DEVELOPERS. *BinPy: An Electronic Simulation Library Written in Pure Python*. 2014. Repositório de código-fonte. Disponível em: <https://github.com/BinPy/BinPy>. Acesso em: 29 out. 2026.

BURCH, C. Logisim: a graphical system for logic circuit design and simulation. *Journal on Educational Resources in Computing*, New York, v. 2, n. 1, p. 5–16, mar. 2002. Disponível em: <https://doi.org/10.1145/545197.545199>. Acesso em: 29 out. 2026.

CARVALHO, L. S. G. de; NAKAMURA, F. G. Práticas de ensino na disciplina de Circuitos Lógicos. *International Journal of Computer Architecture Education (IJCAE)*, Porto Alegre, v. 2, n. 1, p. 9–12, dez. 2013. Disponível em: <https://doi.org/10.5753/ijcae.2013.4943>. Acesso em: 29 out. 2026.

CIRCUITVERSE. *CircuitVerse: Online Digital Logic Circuit Simulator*. 2026. Disponível em: <https://circuitverse.org/>. Acesso em: 29 out. 2026.

CLOW, J. et al. A Pythonic approach for rapid hardware prototyping and instrumentation. In: INTERNATIONAL CONFERENCE ON FIELD PROGRAMMABLE LOGIC AND APPLICATIONS (FPL), 27., 2017, Ghent, Bélgica. *Proceedings [...]*. Piscataway: IEEE, 2017. p. 1–7. Disponível em: <https://doi.org/10.23919/FPL.2017.8056860>. Acesso em: 29 out. 2026.

DECALUWE, J. MyHDL: a Python-based hardware description language. *Linux Journal*, n. 127, nov. 2004. Disponível em: <https://www.linuxjournal.com/article/7542>. Acesso em: 29 out. 2026.

DÔNDICI, F. et al. Um mapeamento sistemático da literatura sobre jogos educacionais digitais para o ensino de Circuitos Lógicos. In: SIMPÓSIO BRASILEIRO DE EDUCAÇÃO EM COMPUTAÇÃO (EDUCOMP), 5., 2025, Juiz de Fora, MG. *Anais [...]*. Porto Alegre: SBC, 2025. p. 240–254. Disponível em: <https://doi.org/10.5753/educomp.2025.5731>. Acesso em: 29 out. 2026.

EBELING, C.; FRENCH, B. Abstract Verilog: a hardware description language for novice students. In: IEEE INTERNATIONAL CONFERENCE ON MICROELECTRONIC SYSTEMS EDUCATION (MSE), 2007, San Diego, EUA. *Proceedings [...]*. Los Alamitos: IEEE Computer Society, 2007. p. 105–106. Disponível em: <https://doi.org/10.1109/MSE.2007.16>. Acesso em: 29 out. 2026.

FLOYD, T. L. *Sistemas Digitais: Fundamentos e Aplicações*. 9. ed. Porto Alegre: Bookman, 2007.

FUKAO, A. T. et al. Estudo sobre evasão nos cursos de Computação da Universidade Estadual de Maringá. In: SIMPÓSIO BRASILEIRO DE EDUCAÇÃO EM COMPUTAÇÃO (EDUCOMP), 3., 2023, On-line. *Anais [...]*. Porto Alegre: SBC, 2023. p. 86–96. Disponível em: <https://doi.org/10.5753/educomp.2023.228209>. Acesso em: 29 out. 2026.

GAMMA, E. et al. *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA: Addison-Wesley, 1995.

GARAY, G. R. et al. Visualization of VHDL-based simulations as a pedagogical tool for supporting computer science education. *Journal of Computational Science*, Amsterdam, v. 36, set. 2019. Art. 100652. Disponível em: <https://doi.org/10.1016/j.jocs.2017.04.004>. Acesso em: 29 out. 2026.

HAASE, J. Flipped classroom with digital circuits: an HTML5-based interactive simulation tool. In: IEEE GLOBAL ENGINEERING EDUCATION CONFERENCE (EDUCON), 2022, Túnis, Tunísia. *Proceedings [...]*. Piscataway: IEEE, 2022. p. 307–312. Disponível em: <https://doi.org/10.1109/EDUCON52537.2022.9766782>. Acesso em: 29 out. 2026.

HACKER, C.; SITTE, R. Interactive teaching of elementary digital logic design with WinLogiLab. *IEEE Transactions on Education*, Piscataway, v. 47, n. 2, p. 196–203, maio 2004. Disponível em: <https://doi.org/10.1109/TE.2004.824843>. Acesso em: 29 out. 2026.

IMAMURA, K. Digital logic design for Computer Science students. *Journal of Computing Sciences in Colleges*, v. 21, n. 1, p. 212–219, out. 2005.

JAMIESON, P. verilogTown – improving students learning hardware description language design – Verilog – with a video game. In: ASEE ANNUAL CONFERENCE & EXPOSITION, 2017, Columbus, EUA. *Proceedings [...]*. Washington, DC: American Society for Engineering Education, 2017. Disponível em: <https://doi.org/10.18260/1-2--29102>. Acesso em: 29 out. 2026.

JURGINA, L. Q. et al. CREsT – uma ferramenta para o auxílio do ensino de confiabilidade em circuitos digitais. In: SIMPÓSIO BRASILEIRO DE INFORMÁTICA NA EDUCAÇÃO (SBIE), 32., 2021, On-line. *Anais [...]*. Porto Alegre: SBC, 2021. p. 191–202. Disponível em: <https://doi.org/10.5753/sbie.2021.218522>. Acesso em: 29 out. 2026.

LABCENTER ELECTRONICS. *Proteus Design Suite*. 2026. Disponível em: <https://www.labcenter.com/simulation/>. Acesso em: 29 out. 2026.

MATERZOK, M. DigitalJS: a visual Verilog simulator for teaching. In: COMPUTER SCIENCE EDUCATION RESEARCH CONFERENCE (CSERC), 8., 2019, Larnaca, Chipre. *Proceedings [...]*. New York: ACM, 2019. p. 110–115. Disponível em: <https://doi.org/10.1145/3375258.3375272>. Acesso em: 29 out. 2026.

MIQUELINI, R. A. A.; FERRARI, H. O. Logisim: ferramenta para simulação de circuitos combinacionais e sequenciais digitais. *Intercursos – Revista Científica*, Ituiutaba, v. 20, n. 2, p. 79–93, 2021. Disponível em: <https://revista.uemg.br/intercursosrevistacientifica/article/view/6319>. Acesso em: 29 out. 2026.

MIRZA, D.; DANGWAL, D.; SHERWOOD, T. PyRTL in early undergraduate research. In: WORKSHOP ON COMPUTER ARCHITECTURE EDUCATION (WCAE), 2019, Phoenix, EUA. *Proceedings [...]*. New York: ACM, 2019. p. 6:1–6:8. Disponível em: <https://doi.org/10.1145/3338698.3338890>. Acesso em: 29 out. 2026.

MYHDL. *What's new in MyHDL 0.6*. 2009. Disponível em: <https://myhdl.readthedocs.io/en/stable/whatsnew/0.6.html>. Acesso em: 29 out. 2026.

SANTINI, L. F. de S. da L. et al. Jogos educativos no ensino de Circuitos Digitais: um mapeamento sistemático. In: SIMPÓSIO BRASILEIRO DE JOGOS E ENTRETENIMENTO DIGITAL (SBGAMES), 22., 2023, Rio Grande, RS. *Anais Estendidos [...]*. Porto Alegre: SBC, 2023. p. 814–825. Disponível em: <https://doi.org/10.5753/sbgames_estendido.2023.234063>. Acesso em: 29 out. 2026.

SAVOLAINEN, V. *Digital Logic Simulator Comparison in Educational Purpose*. Monografia (Bacharelado em Engenharia Eletrônica e de Comunicações) — University of Oulu, Oulu, 2020. Disponível em: <https://urn.fi/URN:NBN:fi:oulu-202005061624>. Acesso em: 29 out. 2026.

SEEMANN, D. G.; BOING, M. *simuGATE*. 2020. Projeto interno do Programa de Educação Tutorial de Engenharia Elétrica (PET EEL) da Universidade Federal de Santa Catarina. Repositório de código-fonte. Disponível em: <https://github.com/miguelboing/simuGATE>. Acesso em: 29 out. 2026.

TOCCI, R. J.; WIDMER, N. S.; MOSS, G. L. *Sistemas Digitais: Princípios e Aplicações*. 11. ed. São Paulo: Pearson Prentice Hall, 2011.